AMRITA VISHWA VIDYAPEETHAM
DEEMED TO BE UNIVERSITY UNDER SECTION 3 OF UGC ACT, 1956

A Simple Structure Constrained Attitude Control for Rigid Bodies

Implementation, reproduction and quadcopter integration of a PD-type prescribed-performance controller

Introduction to Drones – 22AIE448

Group AB02

Nethra R	CB.SC.U4AIE24147
Dheeraj S	CB.SC.U4AIE24050
Raaman Namputhiri	CB.SC.U4AIE24148
Jyothsna	CB.SC.U4AIE24117

School of Artificial Intelligence, Amrita Vishwa Vidyapeetham, Coimbatore

Base paper: M. Golestani et al., *IEEE Access*, vol. 10, pp. 10202–10209, 2022, DOI 10.1109/ACCESS.2022.3144627

What this document is. One place that explains everything in our project: the base paper's controller, how we implemented it, how faithfully we reproduced its simulations (with numbers, not only plots), what we found that the paper does not say, and how we carried the controller onto the Gazebo X3 quadcopter, including why our first version crashed and how it was fixed. Every number in this document is produced by a script in the repository and can be regenerated with the commands of section 2. The equation-by-equation derivations are in the companion document `docs/mathematical_models/Mathematical_Models.pdf`; every MATLAB toy example is in `toy_model/`.

Contents

1 Read Me First **3**
1.1 The project in one paragraph . . . 3
1.2 Results at a glance . . . 3
1.3 The documents and how to read them . . . 4

2 Project Set-up and How to Reproduce Everything **4**
2.1 Repository map . . . 4
2.2 Tools . . . 5
2.3 Commands . . . 5
2.4 Simulation settings common to all results . . . 5
2.5 Metrics (defined once, used everywhere) . . . 6

3 The Controller of the Base Paper, in Code **6**
3.1 Plant and saturation (blocks 1–2) . . . 6
3.2 Envelope (block 3) . . . 7
3.3 Normalised errors, control law and adaptation (blocks 4–6) . . . 7
3.4 One simulation step . . . 8
3.5 The barrier form: paper versus symmetric . . . 9

4 Reproduction of the Paper's Simulations **9**
4.1 Part 1: effect of the settling time T_s . . . 9
4.2 Part 2: constrained PD-type versus conventional PD . . . 11
4.3 Adaptive estimate and Lyapunov function . . . 14
4.4 Independent cross-check in MATLAB . . . 14

5 Beyond the Paper: Robustness, Sensitivity and Findings **15**
5.1 Monte-Carlo studies . . . 15
5.2 Sensitivity to the parameters the paper does not give . . . 17
5.3 Findings: what the paper does not say . . . 17

6 Putting the Controller on the Gazebo X3 Quadcopter **18**
6.1 Architecture . . . 18
6.2 Why the first Gazebo node crashed . . . 19
6.3 The corrected controller (v2) . . . 20
6.4 The ROS 2 node . . . 21

7 Results on the X3 **22**
7.1 E0/E1: the crash of v1, reproduced and explained . . . 22
7.2 E2: 180 s hover with the paper's law . . . 23

7.3 E3: disturbance rejection, constrained vs. conventional PD 24
7.4 E4: square trajectory 26
7.5 E5: control rate, delay and jitter 26

8 Visual Demonstrations (MuJoCo and Browser) 27
8.1 MuJoCo: two bodies side by side 27
8.2 MuJoCo: flight with altitude and attitude loops 27
8.3 Interactive browser simulator 28

9 The MATLAB Toy Model 28
9.1 The files 29
9.2 How to run them 29
9.3 What the steps show 29
9.4 Printed output 31
9.5 Things to try 31

10 Conclusions and Scope of the Claims 31
10.1 What we have shown 31
10.2 What we do not claim 32
10.3 Conclusion 33
10.4 Future work 33

A Appendix 33
A.1 Where every figure and number comes from 33
A.2 All parameters in one place 34
A.3 References 34

1 Read Me First

1.1 The project in one paragraph

The base paper (Golestani et al., IEEE Access 2022) proposes a PD-type attitude controller that keeps *both* the attitude quaternion and the angular velocity of a rigid body inside time-varying bounds that close in a finite, chosen time, while the actuators saturate and an unknown disturbance acts. Its selling point is simplicity: an ordinary PD law plus one barrier-weighted damping term and one scalar adaptive estimate. We (1) implemented every equation of the paper, (2) reproduced both of its simulation studies quantitatively, (3) wrote an independent MATLAB toy model that re-derives every block and re-implements the experiments, (4) tested the controller beyond the paper (300-run Monte-Carlo studies, parameter sensitivity, an alternative barrier form), and (5) integrated it on the Gazebo X3 quadcopter through ROS 2, where we found and fixed the faults that made our first version crash.

1.2 Results at a glance

Question	Answer (measured)	Where
Does our implementation reproduce the paper's Part 1?	Yes. For $T_s = 15\,\mathrm{s}$ and $10\,\mathrm{s}$ both $\boldsymbol{q}_v$ and $\boldsymbol{\omega}$ stay strictly inside their envelopes (0 % of the time outside; peak $\lvert\varepsilon\rvert = 0.871/0.912$ and $0.924/0.942$), and the shorter T_s converges faster (settling 5.25 s vs. 4.53 s), as the paper states.	section 4.1
Does it reproduce Part 2 (constrained vs. conventional PD)?	Yes. Same gains, limit, disturbance and initial state: the constrained law never leaves the envelopes; conventional PD leaves the angular-velocity envelope 27.5 % of the time. The constrained law also uses 36 % less torque energy and halves the final rate error.	section 4.2
Is this a one-off?	No. 300 random initial states in the paper's regime: 97.0 % violation-free (95 % CI 94.4–98.4) for the constrained law, 0.0 % for PD.	section 5.1
Can it be checked independently?	The MATLAB toy model re-implements the spacecraft experiments separately (`toy_model/step6`); `docs/make_crosscheck.py` puts its numbers next to the Python ones (table 3).	section 4.4
What did we find that the paper does not say?	Four unstated parameters; Part 2 starts on its own boundary; the barrier is one-sided; the envelope must be feasible for the actuator; Theorem 1 proves convergence, not envelope invariance.	section 5.3
Does it fly a quadcopter?	Yes, after fixing our first node. The paper's law, re-sized for the X3, hovers for 180 s and flies a $2 \times 2\,\mathrm{m}$ square (4.0 cm RMS error) on a software-in-the-loop model built from Gazebo's X3 files, with 0 % attitude-envelope violations. The ROS 2 node `gazebo.py` runs exactly this controller.	section 7
Why did the first Gazebo version crash?	Two faults, isolated one by one: a mixer whose signs did not match the X3 (no pitch authority) and an altitude-rate estimate that was never updated. Fixing both gives a stable flight.	section 6.2

1.3 The documents and how to read them

Document	Content
`docs/complete_documentation/` (this PDF)	project, results, code walk-through, X3 integration, claims
`docs/mathematical_models/`	every equation: boxed statement, symbol table, derivation, hand-checkable toy example, flow diagrams
`toy_model/`	MATLAB scripts (`step1`–`step8`) that print the toy examples and plot each idea; `run_all.m` runs all of them
`docs/results/*.md, *.json`	every number of this document, written by the scripts
`docs/figures/`	every figure of this document, written by the scripts

Colour code of the boxes. Blue: the idea in plain words. Orange: a toy example with numbers. Green: what it means in our project. Red: something to be careful about.

Naming of the controllers. Throughout, *constrained (paper Eq. 13)* is the paper's law exactly as printed; *constrained (symmetric barrier)* is our variant with $(1-\varepsilon^2)^{-2}$ in place of $(1-\varepsilon)^{-2}$ (section 3.5); *conventional PD* is the same law with $\boldsymbol{u}_{cons} = 0$, i.e. the paper's Part 2 baseline.

2 Project Set-up and How to Reproduce Everything

2.1 Repository map

Path	What it is
`base paper.pdf`	the base paper
`paper_reproduction.py`	reproduction of the paper's Section IV (Part 1, Part 2), sensitivity, Monte-Carlo; writes `docs/figures/fig*.png`, `docs/results/metrics.*`
`x3_sim/x3_model.py`	software-in-the-loop surrogate of the Gazebo X3 (all constants from the Gazebo model files)
`x3_sim/controllers.py`	`OriginalGazeboController` (exact port of our first node) and `FixedController` (the corrected paper controller)
`x3_sim/run_x3_experiments.py`	experiments E0–E5 on the X3; writes `docs/figures/x3_*.png`, `docs/results/x3_metrics.*`
`gazebo.py`	ROS 2 node (v2) that flies the Gazebo X3 with `FixedController`
`plot_gazebo_log.py`	turns a log of `gazebo.py` into figures and metrics
`tests/test_gazebo_node_offline.py`	runs the real `gazebo.py` without ROS against the surrogate
`toy_model/`	MATLAB toy model, steps 1–8, `run_all.m`, `lib/`
`Drones_Mujoco_*.ipynb`, `One_drone_alligning.ipynb`	MuJoCo 3-D attitude and flight demos
`index.html`, `app.js`, `simulation.js`, `style.css`	interactive browser simulator (illustration)
`legacy/`	first versions kept for the record (`gazebo_v1.py`, early drafts); not used as evidence
`docs/`	this document, the mathematical models, all figures and results

2.2 Tools

Tool	Used for
Python 3 + NumPy + Matplotlib	reproduction, X3 surrogate, plots (no other dependency)
MATLAB (no toolbox)	toy model and independent cross-check of the reproduction
ROS 2 + Gazebo Sim (gz)	X3 quadcopter world; `ros_gz_bridge` for `/world/quadcopter/dynamic_pose/info` and `/X3/gazebo/command/motor_speed`
MuJoCo (Python)	3-D visual demos of the attitude controller
LaTeX (`pdflatex`)	both documents

2.3 Commands

```
pip install numpy matplotlib
python3 paper_reproduction.py            # Part 1, Part 2, sensitivity, Monte-Carlo  (~6 min)
python3 x3_sim/run_x3_experiments.py     # X3 experiments E0-E5                      (~16 min on 2
     cores)
python3 tests/test_gazebo_node_offline.py  # runs gazebo.py against the X3 surrogate (~2 min)
```

In MATLAB, from any folder:

```
run('<repo>/toy_model/run_all.m')
```

It writes `toy_model/figures/*.png` and the complete printed output to `toy_model/results_log.txt`.

Gazebo (with the `quadcopter` world running and the two topics bridged):

```
python3 gazebo.py --ros-args -p controller_mode:=constrained -p experiment:=hover -p duration
    :=180.0
python3 gazebo.py --ros-args -p experiment:=square -p duration:=45.0
python3 gazebo.py --ros-args -p controller_mode:=pd -p disturbance_scale:=0.5
python3 plot_gazebo_log.py /tmp/drone_constrained_hover.npz
```

In our project — reproducibility

Running `paper_reproduction.py` again on a clean checkout reproduced `docs/results/metrics.md` and `metrics.json` *bit for bit* (the Monte-Carlo uses a fixed seed, 7). The X3 experiments are deterministic as well (fixed seeds), and the offline test of `gazebo.py` passes (hover ends at $z = 2.000\,\mathrm{m}$, the square returns to $(0, 0, 2)\,\mathrm{m}$).

2.4 Simulation settings common to all results

Setting	Value
Integrator	fixed-step RK4, $\Delta t = 1\,\mathrm{ms}$; quaternion re-normalised after every step
Control	computed once per step and held (zero-order hold), as in a digital controller
Saturation	Eq. (3)–(4), applied to the commanded torque
Barrier	Eq. (22) (boundary layer), ε fed to the barrier limited to 0.999 (never active inside the envelope)
Adaptive law	explicit Euler, $\hat{\theta} \geq 0$
Unstated parameters	$c = 0.5$, $\varsigma = 0.02$, Part 2: $T_s = 15\,\mathrm{s}$, $\rho_{q0} = 0.36$ (Mathematical Models, Section 2)
Horizon	20 s (the paper's plots)

2.5 Metrics (defined once, used everywhere)

With T the simulation length and $\mathbb{1}[\cdot]$ the indicator function:

$$V_q = \frac{100}{T}\int_0^T \mathbb{1}\big[\exists i : |q_i| \geq \rho_q\big]\,dt, \qquad V_\omega = \frac{100}{T}\int_0^T \mathbb{1}\big[\exists i : |\omega_i| \geq \rho_\omega\big]\,dt \qquad (\% \text{ of time outside}),$$

$$\varepsilon_{\max} = \max_{t,i} |\varepsilon_{li}(t)|, \qquad S_u = \frac{100}{T}\int_0^T \mathbb{1}\big[\exists i : |u_i| \geq u_{\max}\big]\,dt, \qquad E_u = \int_0^T \|\operatorname{sat}(\boldsymbol{u})\|^2 dt,$$

plus the settling time (last instant with $\|\boldsymbol{q}_v\| > 0.01$), the final $\|\boldsymbol{q}_v\|$ and $\|\boldsymbol{\omega}\|$. A run is *violation-free* if $V_q = V_\omega = 0$.

3 The Controller of the Base Paper, in Code

This section connects each equation of the paper to the lines of `paper_reproduction.py` that implement it. The derivations and the hand-checkable examples are in the Mathematical Models document; here we only state the equations again so that the code can be read next to them.

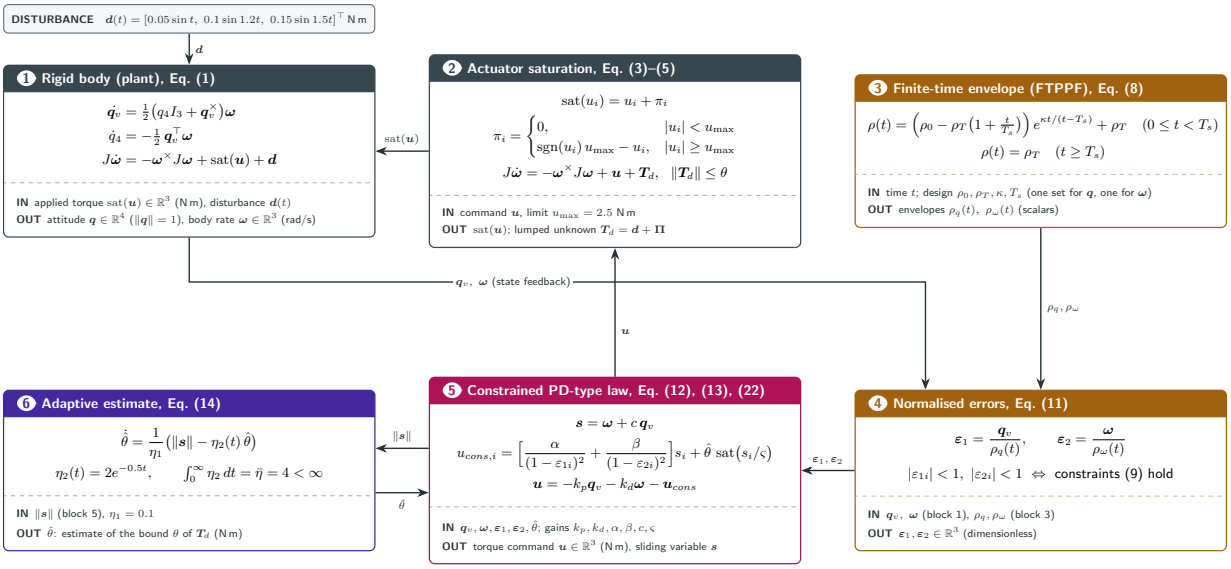


Figure 1: Equation flow of the paper. Block numbers are used throughout this document.

3.1 Plant and saturation (blocks 1–2)

Eq. (1), (3)–(5)

$$\dot{\boldsymbol{q}}_v = \tfrac{1}{2}(q_4 I_3 + \boldsymbol{q}_v^\times)\boldsymbol{\omega}, \qquad \dot{q}_4 = -\tfrac{1}{2}\boldsymbol{q}_v^\top\boldsymbol{\omega}, \qquad J\dot{\boldsymbol{\omega}} = -\boldsymbol{\omega}^\times J\boldsymbol{\omega} + \operatorname{sat}(\boldsymbol{u}) + \boldsymbol{d}$$
$$\operatorname{sat}(u_i) = \operatorname{sgn}(u_i)\min(|u_i|, u_{\max})$$

The simulator is written for a whole *batch* of rigid bodies at once (rows of the arrays), which is what makes a 300-run Monte-Carlo study take seconds instead of hours.

paper_reproduction.py — rhs(): Eq. (1) for a batch

```
107 def rhs(q, w, u, d):
108     """Eq. (1): returns q_dot (N,4) and w_dot (N,3) for a batch."""
109     qv, q4 = q[:, :3], q[:, 3:4]
110     qv_dot = 0.5 * (q4 * w + np.cross(qv, w))
```

```
    q4_dot = -0.5 * np.sum(qv * w, axis=1, keepdims=True)
    Jw = w @ J.T
    w_dot = (-np.cross(w, Jw) + u + d) @ J_INV.T
    return np.hstack([qv_dot, q4_dot]), w_dot
```

3.2 Envelope (block 3)

Eq. (8)

$$\rho(t) = \Big(\rho_0 - \rho_T\big(1 + \tfrac{t}{T_s}\big)\Big)\exp\Big(\frac{\kappa t}{t - T_s}\Big) + \rho_T \quad (0 \le t < T_s), \qquad \rho(t) = \rho_T \quad (t \ge T_s)$$

paper_reproduction.py — ftppf(): Eq. (8)

```
def ftppf(t, rho0, rhoT, kappa, Ts):
    """Finite-time prescribed performance function, Eq. (8). Works on arrays."""
    t = np.asarray(t, dtype=float)
    out = np.full_like(t, rhoT)
    m = t < Ts
    tm = t[m]
    out[m] = (rho0 - rhoT * (1.0 + tm / Ts)) * np.exp(kappa * tm / (tm - Ts)) + rhoT
    return out
```

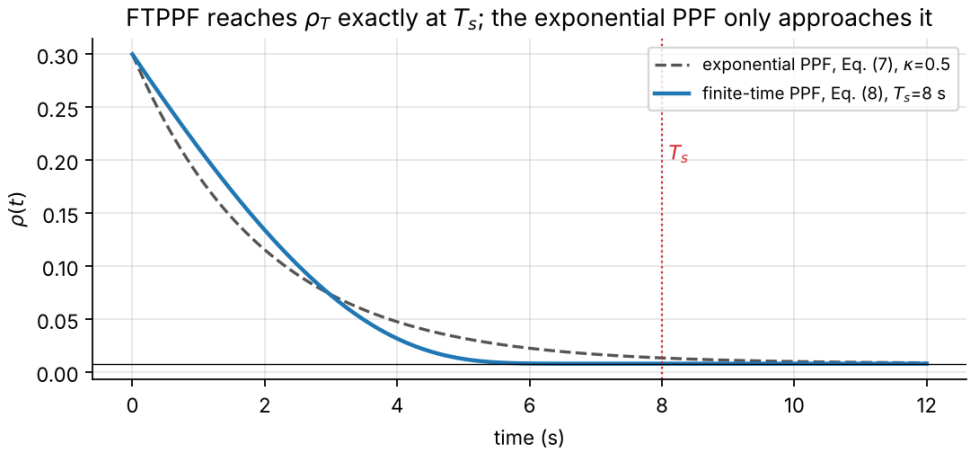


Figure 2: The finite-time property (cf. paper Fig. 2): the FTPPF reaches ρ_T exactly at $T_s = 8\,\mathrm{s}$ and stays there; the exponential PPF is still $1.67\,\rho_T$ at that time.

3.3 Normalised errors, control law and adaptation (blocks 4–6)

Eq. (11), (12), (22), (14)

$$\varepsilon_1 = \frac{\boldsymbol{q}_v}{\rho_q}, \qquad \varepsilon_2 = \frac{\boldsymbol{\omega}}{\rho_\omega}, \qquad \boldsymbol{s} = \boldsymbol{\omega} + c\boldsymbol{q}_v, \qquad \dot{\hat{\theta}} = \frac{\|\boldsymbol{s}\| - \eta_2\hat{\theta}}{\eta_1}$$

$$u_i = -k_p q_i - k_d\omega_i - \Big(\Big[\frac{\alpha}{(1-\varepsilon_{1i})^2} + \frac{\beta}{(1-\varepsilon_{2i})^2}\Big]s_i + \hat{\theta}\,\mathrm{sat}(s_i/\varsigma)\Big)$$

paper_reproduction.py — control(): Eq. (11), (12), (22) and the two comparison controllers

```
def control(t, q, w, th, p, mode):
    """Eq. (12)-(13) with Eq. (22) boundary layer. Returns u (unsaturated), s, eps1, eps2."""
    qv = q[:, :3]
```

```
    rq = ftppf(np.array([t]), p["rho_q0"], p["rho_qT"], p["kappa_q"], p["Ts"])[0]
    rw = ftppf(np.array([t]), p["rho_w0"], p["rho_wT"], p["kappa_w"], p["Ts"])[0]
    e1 = qv / rq
    e2 = w / rw
    s = w + p["c"] * qv
    u = -p["kp"] * qv - p["kd"] * w
    if mode == "constrained":            # paper form, Eq. (13)
        e1c = np.minimum(e1, 0.999)       # numerical guard only (never active when |eps|<1)
        e2c = np.minimum(e2, 0.999)
        gain = p["alpha"] / (1 - e1c) ** 2 + p["beta"] / (1 - e2c) ** 2
        u = u - (gain * s + th[:, None] * np.clip(s / p["varsigma"], -1, 1))
    elif mode == "symmetric":            # variant used in the original browser/Gazebo code
        e1c = np.clip(e1, -0.999, 0.999)
        e2c = np.clip(e2, -0.999, 0.999)
        gain = p["alpha"] / (1 - e1c ** 2) ** 2 + p["beta"] / (1 - e2c ** 2) ** 2
        u = u - (gain * s + th[:, None] * np.clip(s / p["varsigma"], -1, 1))
    elif mode == "pd":
        pass
    else:
        raise ValueError(mode)
    return u, s, e1, e2, rq, rw
```

3.4 One simulation step

Per step of 1 ms: compute the control from the current state, saturate it, update all metrics, integrate the plant with RK4 holding the torque constant, re-normalise the quaternion, update $\hat{\theta}$.

paper_reproduction.py — simulate(): the main loop

```
    for k in range(n):
        t = k * dt
        u, s, e1, e2, rq, rw = control(t, q, w, th, p, mode)
        us = sat_vec(u, p["umax"])
        # metrics
        vq = np.any(np.abs(q[:, :3]) >= rq, axis=1)
        vw = np.any(np.abs(w) >= rw, axis=1)
        viol_q += vq; viol_w += vw
        at_lim += np.any(np.abs(u) >= p["umax"], axis=1)
        emax1 = np.maximum(emax1, np.max(np.abs(e1), axis=1))
        emax2 = np.maximum(emax2, np.max(np.abs(e2), axis=1))
        energy += np.sum(us ** 2, axis=1) * dt
        upeak = np.maximum(upeak, np.max(np.abs(u), axis=1))
        last_out = np.where(np.linalg.norm(q[:, :3], axis=1) > 0.01, t, last_out)
        if record and k % rec_every == 0:
            L["t"].append(t); L["q"].append(q.copy()); L["w"].append(w.copy())
            L["u"].append(u.copy()); L["usat"].append(us.copy()); L["th"].append(th.copy())
            L["rq"].append(rq); L["rw"].append(rw); L["e1"].append(e1.copy())
            L["e2"].append(e2.copy()); L["s"].append(s.copy())
        # RK4 with ZOH torque; disturbance evaluated at stage times
        def f(tt, qq, ww):
            d = disturbance(tt)[None, :] * ds
            return rhs(qq, ww, us, d)
        k1q, k1w = f(t, q, w)
        k2q, k2w = f(t + dt / 2, q + dt / 2 * k1q, w + dt / 2 * k1w)
        k3q, k3w = f(t + dt / 2, q + dt / 2 * k2q, w + dt / 2 * k2w)
        k4q, k4w = f(t + dt, q + dt * k3q, w + dt * k3w)
        q = q + dt / 6 * (k1q + 2 * k2q + 2 * k3q + k4q)
        w = w + dt / 6 * (k1w + 2 * k2w + 2 * k3w + k4w)
        q = q / np.linalg.norm(q, axis=1, keepdims=True)
        if mode != "pd":
            # adaptive law, Eq. (14) (explicit Euler; theta_hat >= 0 projection)
            th = np.maximum(th + dt * (np.linalg.norm(s, axis=1) - eta2_fun(t) * th) / p["eta1"],
    0.0)
```

3.5 The barrier form: paper versus symmetric

One-sided or two-sided?

The paper writes the barrier as $\alpha(1-\varepsilon_{1i})^{-2}$. Read literally, this gain explodes when $\varepsilon_{1i} \to +1$ but tends to $\alpha/4$ when $\varepsilon_{1i} \to -1$: it guards the upper wall of the tube far more than the lower wall. The two-sided version $\alpha(1-\varepsilon_{1i}^2)^{-2}$ treats both walls alike. Our first browser and Gazebo code used the two-sided version without saying so. We now treat the paper form as the reference and report the symmetric form as a named variant in every table.

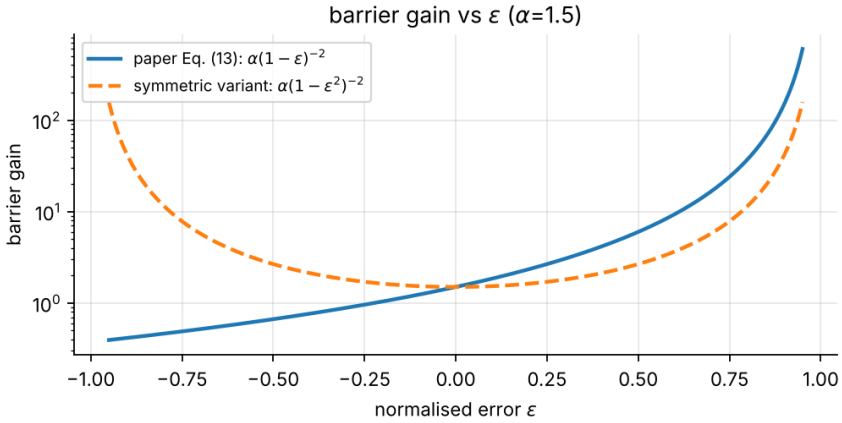


Figure 3: Barrier gain as a function of ε for $\alpha = 1.5$.

Toy example — the same state, two forms

Part-1 state at $t = 0$, axis 3: $\varepsilon_{13} = -0.833$ (closest to its wall of the three axes). Paper form: $1.5/(1+0.833)^2 = 0.446$, plus the rate part, $g_3 = 1.59$. Symmetric form: $1.5/(1-0.694)^2 = 16.0$, $g_3 = 17.3$. The paper form gives the axis that is closest to the wall the *smallest* gain. (MATLAB: `step2_normalised_error.m`.)

4 Reproduction of the Paper's Simulations

The paper validates its controller on a rigid spacecraft in two parts (its Section IV). We repeated both with the paper's inertia, disturbance, actuator limit, gains, envelopes and initial states (Mathematical Models, Section 2.3). The paper publishes plots only, so we compare the behaviour of the plots and, in addition, report the metrics of section 2.5, which make the comparison checkable. All numbers: `docs/results/metrics.md`.

4.1 Part 1: effect of the settling time T_s

Paper's claim. The quaternion and the angular velocity stay inside $\pm\rho(t)$, the controller does not let the trajectories touch the boundaries, a smaller T_s gives faster convergence, and the torque stays within the limit.

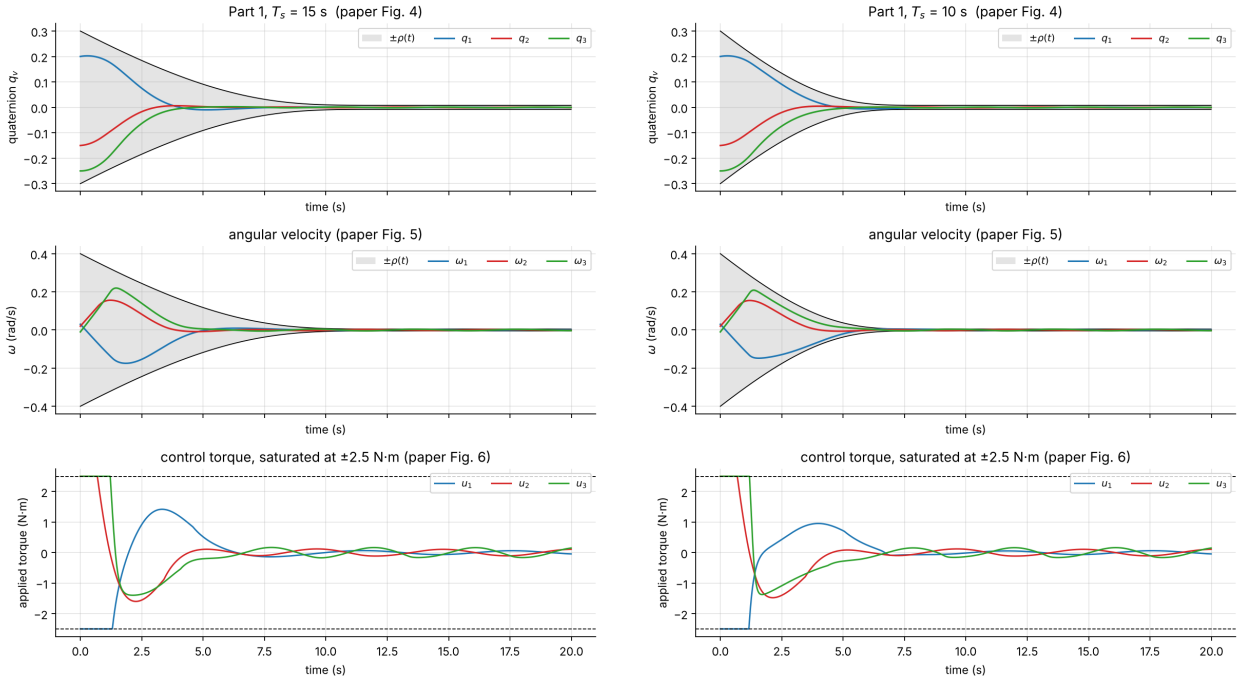


Figure 4: Part 1 reproduction, $T_s = 15\,\mathrm{s}$ (left) and $T_s = 10\,\mathrm{s}$ (right). Top: $\boldsymbol{q}_v$ inside $\pm\rho_q$ (paper Fig. 4). Middle: $\boldsymbol{\omega}$ inside $\pm\rho_\omega$ (paper Fig. 5). Bottom: applied torque, saturated at $\pm 2.5\,\mathrm{N\,m}$ during the first second (paper Fig. 6).

Table 1: Part 1 metrics (constrained law, paper form).

Metric	$T_s = 15\,\mathrm{s}$	$T_s = 10\,\mathrm{s}$
time outside ρ_q / outside ρ_ω	0 % / 0 %	0 % / 0 %
$\max\lvert\varepsilon_1\rvert$ / $\max\lvert\varepsilon_2\rvert$	0.871 / 0.912	0.924 / 0.942
settling time ($\lVert\boldsymbol{q}_v\rVert < 0.01$)	5.25 s	4.53 s
final $\lVert\boldsymbol{q}_v\rVert$ / final $\lVert\boldsymbol{\omega}\rVert$ (rad/s)	4.4×10^{-4} / 4.0×10^{-3}	4.5×10^{-4} / 4.1×10^{-3}
time with commanded torque at the limit	6.5 %	5.9 %
peak commanded torque (before saturation)	10.54 N m	10.54 N m
$\hat{\theta}$ at 20 s	0.433	0.416

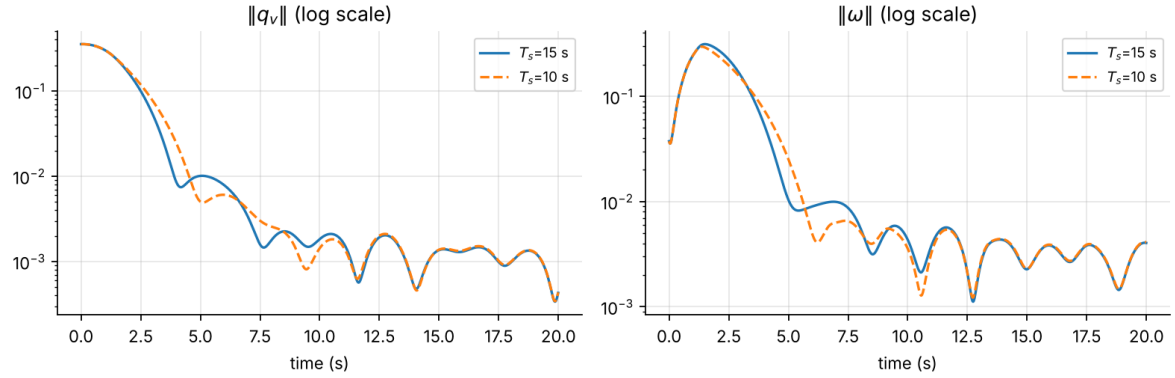


Figure 5: Convergence on a log scale: with $T_s = 10\,\mathrm{s}$ both $\lVert\boldsymbol{q}_v\rVert$ and $\lVert\boldsymbol{\omega}\rVert$ fall faster during the transient. After the envelope has closed, both runs settle on the same small residual set by the disturbance and the boundary layer.

In our project — result

Every statement of Part 1 is reproduced: 0 % of the time outside either envelope for both settling times, the largest normalised error is 0.942 (6 % margin to the wall), the shorter T_s settles 0.72 s earlier, and the applied torque never exceeds 2.5 N m (it saturates only in the first ≈ 1.2 s, when the initial error is largest). The peak of the *commanded* torque (10.54 N m, at $t = 0$) is the value computed by hand in the Mathematical Models (block 5).

4.2 Part 2: constrained PD-type versus conventional PD

Paper's claim. With the same maximum control torque, the constrained law keeps both states inside their envelopes, whereas conventional PD is "not capable of providing as fast and accurate convergence".

Set-up. Both controllers have the same $k_p = 40$, $k_d = 20$, the same 2.5 N m limit, the same disturbance and the same initial state $\boldsymbol{q}(0) = [0.3, -0.2, -0.3, 0.8832]$, $\boldsymbol{\omega}(0) = [-0.02, -0.02, 0.025]$ rad/s; conventional PD is the law with $\boldsymbol{u}_{cons} = 0$. $T_s = 15$ s and $\rho_{q0} = 0.36$ (the paper's 0.3 would put the initial state on the boundary).

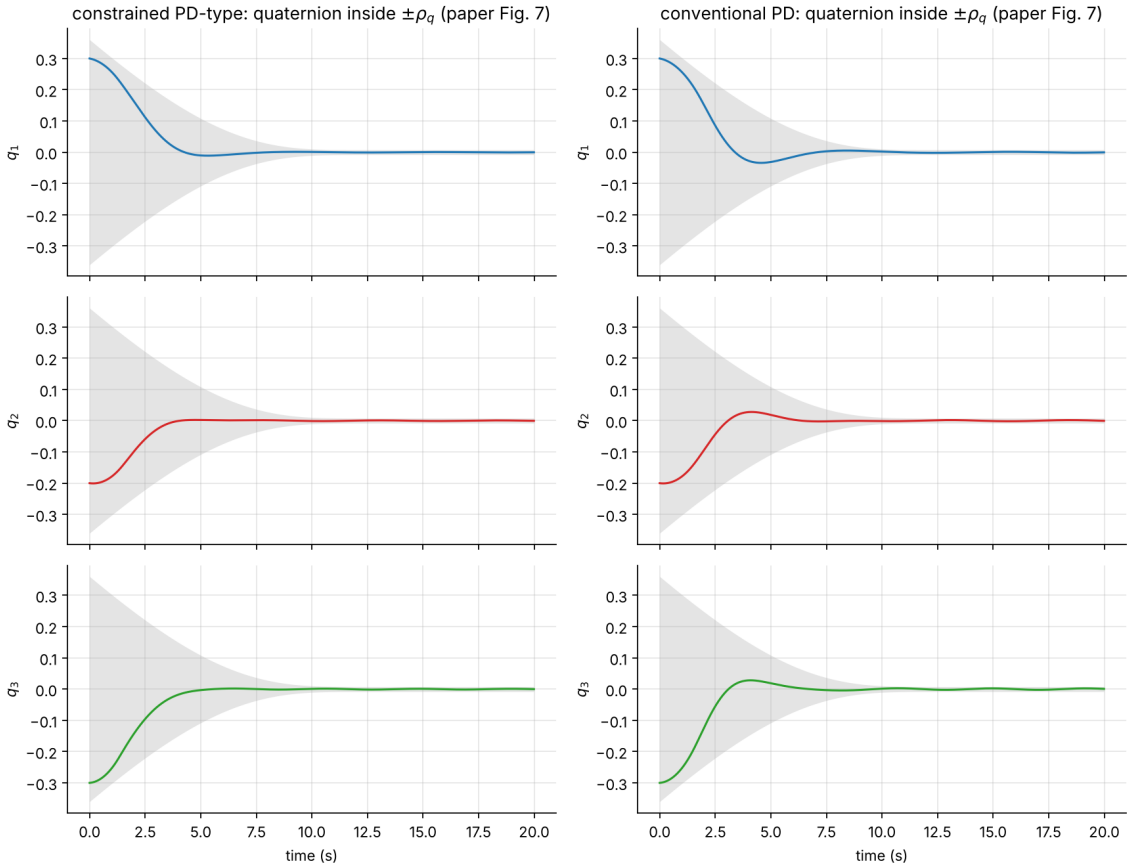


Figure 6: Part 2, quaternion (paper Fig. 7). Both controllers keep $\boldsymbol{q}_v$ inside $\pm\rho_q$ in this run.

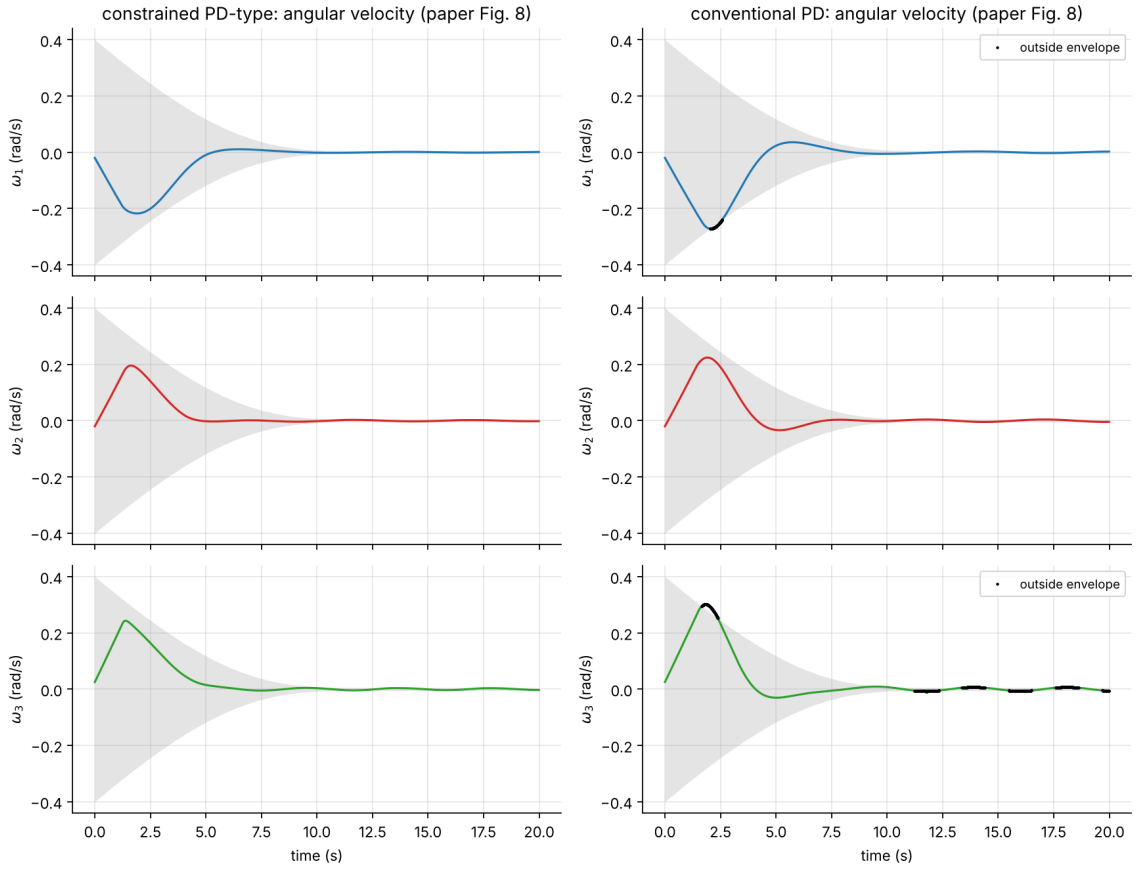


Figure 7: Part 2, angular velocity (paper Fig. 8). Black dots: conventional PD outside $\pm\rho_\omega$, both during the transient ($t \approx 2\,\mathrm{s}$, axes 1 and 3) and in steady state, where the disturbance keeps $|\omega_3|$ above $\rho_{\omega T} = 0.005\,\mathrm{rad/s}$. The constrained law stays inside throughout.

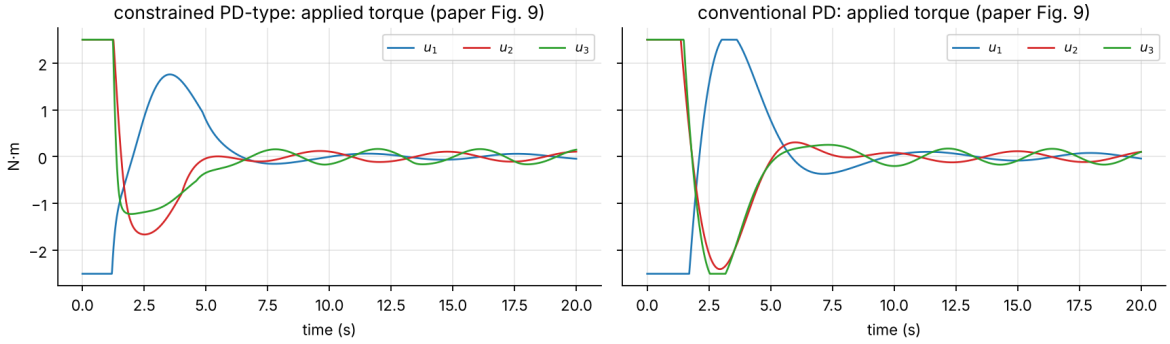


Figure 8: Part 2, applied torque (paper Fig. 9). Both controllers use the same maximum torque, $2.5\,\mathrm{N\,m}$, as in the paper.

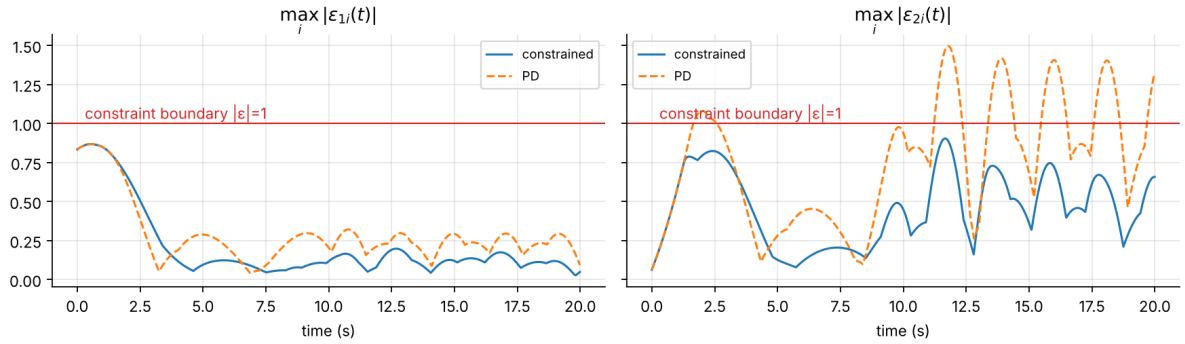


Figure 9: The clearest view of the constraint: $\max_i |\varepsilon_{1i}|$ and $\max_i |\varepsilon_{2i}|$. Below the red line means inside. Conventional PD crosses it for the angular velocity.

Table 2: Part 2 metrics ($T_s = 15\,\mathrm{s}$, $\rho_{q0} = 0.36$; same gains, limit, disturbance and initial state).

Metric	constrained (paper)	constrained (symm.)	conventional PD
time outside ρ_q	0 %	0 %	0 %
time outside ρ_ω	**0 %**	**0 %**	**27.5 %**
$\max \lvert\varepsilon_1\rvert$ / $\max \lvert\varepsilon_2\rvert$	0.868 / 0.904	0.868 / 0.837	0.868 / 1.496
time with commanded torque at the limit	6.3 %	7.2 %	14.0 %
peak commanded torque	18.8 N m	13.8 N m	11.6 N m
settling time ($\lVert\boldsymbol{q}_v\rVert < 0.01$)	5.79 s	4.19 s	6.31 s
final $\lVert\boldsymbol{\omega}\rVert$ (rad/s)	0.0040	0.0040	0.0083
final $\lVert\boldsymbol{q}_v\rVert$	4.4×10^{-4}	3.9×10^{-4}	9.0×10^{-4}
torque energy $\int \lVert \mathrm{sat}(\boldsymbol{u})\rVert^2 dt$	38.9	41.5	60.5
final $\hat{\theta}$	0.439	0.435	–

In our project — result

The central claim of the paper is reproduced. At the same 2.5 N m limit the constrained law keeps the angular velocity inside its envelope all the time, conventional PD is outside 27.5 % of the time (peak $|\varepsilon_2| = 1.50$). The constrained law also settles earlier (5.79 vs. 6.31 s), ends with half the rate error (0.0040 vs. 0.0083 rad/s) and less than half the attitude error, spends less than half as much time at the torque limit (6.3 vs. 14.0 %) and uses 36 % less torque energy.

Reading Part 2 fairly

In this run the conventional PD also keeps the *quaternion* inside its envelope; the advantage of the constrained law is in the angular velocity, the residual error and the control effort. The constrained law commands a higher *peak* torque (18.8 N m) for an instant, but the actuator delivers at most 2.5 N m to both, which is exactly the paper's "same maximum control effort".

4.3 Adaptive estimate and Lyapunov function

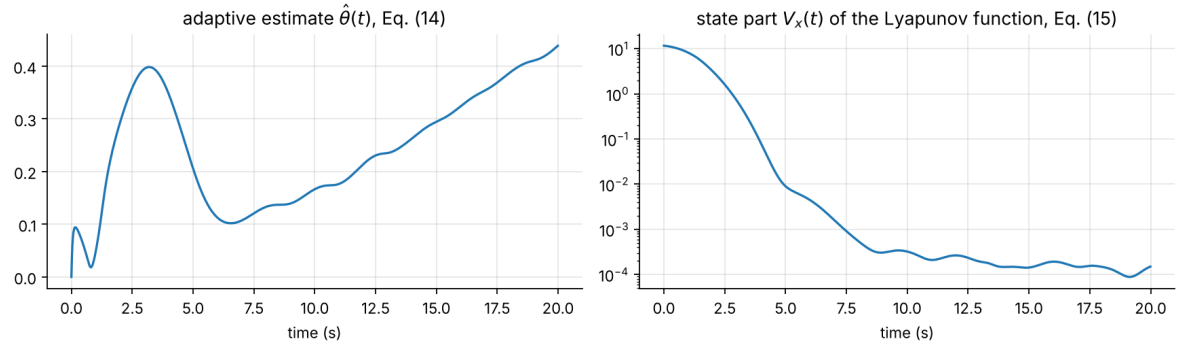


Figure 10: Left: $\hat{\theta}(t)$, Eq. (14). It rises while $\|\boldsymbol{s}\|$ is large, drops when the leakage η_2 is still strong, and then creeps upwards because $\eta_2 = 2e^{-0.5t}$ has almost vanished while a small residual $\|\boldsymbol{s}\|$ remains (Mathematical Models, block 6). Right: the state part V_x of the Lyapunov function (15) along the Part 2 trajectory. It falls by five orders of magnitude within 9 s and then stays near 10^{-4}, a floor set by the persistent disturbance and the boundary layer (ultimate boundedness rather than exact convergence; Mathematical Models, Theorem 1). The $\tilde{\theta}$ part of (15) cannot be plotted because the true bound θ is unknown.

4.4 Independent cross-check in MATLAB

`toy_model/step6_spacecraft_closed_loop.m` is a second implementation of the same experiments in a different language, with its own RK4 integrator and its own control law (`lib/control_law.m`); only the equations and parameters are shared. **The MATLAB column is empty because `toy_model/run_all.m` has not been run on this checkout.** Run it in MATLAB, then `python3 docs/make_crosscheck.py`, and recompile.

Table 3: Cross-check: Python (`paper_reproduction.py`) vs. MATLAB (`toy_model/step6`).

Case	Metric	Python	MATLAB
Part 1, $T_s = 15\,\mathrm{s}$	$\max\lvert\varepsilon_1\rvert$	0.871	–
	$\max\lvert\varepsilon_2\rvert$	0.912	–
	time outside ρ_q (%)	0.0	–
	time outside ρ_ω (%)	0.0	–
	settling time (s)	5.25	–
Part 1, $T_s = 10\,\mathrm{s}$	$\max\lvert\varepsilon_1\rvert$	0.924	–
	$\max\lvert\varepsilon_2\rvert$	0.941	–
	time outside ρ_q (%)	0.0	–
	time outside ρ_ω (%)	0.0	–
	settling time (s)	4.53	–
Part 2, paper form	$\max\lvert\varepsilon_2\rvert$	0.904	–
	time outside ρ_ω (%)	0.0	–
	time at torque limit (%)	6.3	–
	settling time (s)	5.79	–
	torque energy	38.9	–
Part 2, symmetric	$\max\lvert\varepsilon_2\rvert$	0.837	–
	time outside ρ_ω (%)	0.0	–
	time at torque limit (%)	7.2	–
	settling time (s)	4.19	–
	torque energy	41.5	–
Part 2, conv. PD	$\max\lvert\varepsilon_2\rvert$	1.496	–
	time outside ρ_ω (%)	27.5	–
	time at torque limit (%)	14.0	–
	settling time (s)	6.31	–
	torque energy	60.5	–

5 Beyond the Paper: Robustness, Sensitivity and Findings

The paper shows two single runs. To know whether its conclusions hold in general, and where they stop holding, we ran three Monte-Carlo studies (300 runs each, the same random draws for all three controllers, fixed seed) and a sensitivity study on the parameters the paper does not state.

5.1 Monte-Carlo studies

Study	Random draws (per run)
A — paper regime	attitude uniform inside 90 % of the envelope ($\lvert q_i(0)\rvert \le 0.9\rho_{q0}$), $\lvert\omega_i(0)\rvert \le 0.05\,\mathrm{rad/s}$, paper disturbance
B — disturbance amplitude	as A, disturbance multiplied by a factor drawn from $U[0.5, 3]$
C — initial-rate stress	as A, but $\lvert\omega_i(0)\rvert$ up to $0.9\rho_{\omega 0} = 0.36\,\mathrm{rad/s}$

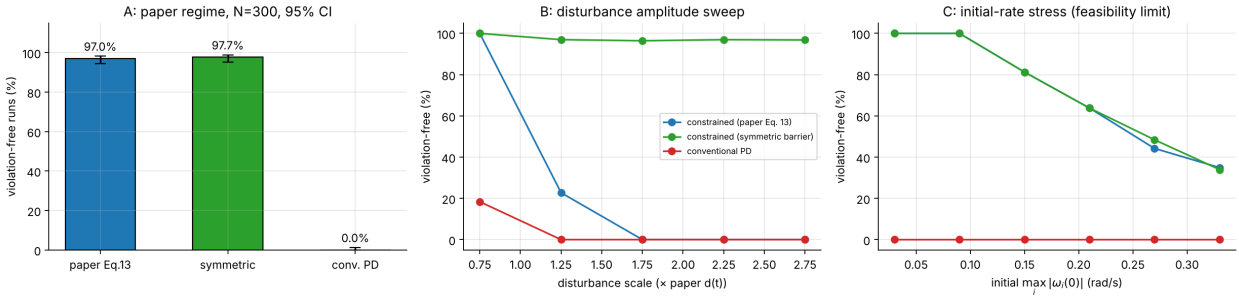


Figure 11: Fraction of violation-free runs. A: paper regime with 95 % Wilson confidence intervals. B: by disturbance scale. C: by initial rate.

Table 4: Monte-Carlo results ($N = 300$ per controller and study). "Violation-free" = never outside either envelope.

Study	Controller	violation-free	95 % CI	median settling	mean torque energy
A	constrained (paper)	**97.0 %**	[94.4, 98.4]	4.85 s	27.9
	constrained (symmetric)	97.7 %	[95.3, 98.9]	4.89 s	28.5
	conventional PD	0.0 %	[0.0, 1.3]	5.93 s	36.0
B	constrained (paper)	21.3 %	[17.1, 26.3]	5.05 s	29.1
	constrained (symmetric)	**97.3 %**	[94.8, 98.6]	5.03 s	29.8
	conventional PD	3.0 %	[1.6, 5.6]	6.00 s	37.2
C	constrained (paper)	50.3 %	[44.7, 56.0]	6.72 s	76.5
	constrained (symmetric)	51.3 %	[45.7, 56.9]	6.68 s	89.6
	conventional PD	0.0 %	[0.0, 1.3]	7.65 s	59.3

In our project — what the three studies say

1. **A confirms the paper in general, not just for its two initial states.** In the paper's operating regime 97 % of all random starts stay inside both envelopes with the paper's law; conventional PD fails in every run (it leaves the rate envelope both during the transient and after it has closed to $\rho_{\omega T} = 0.005\,\mathrm{rad/s}$, where the disturbance keeps its residual rate too large). The constrained law also settles about 1 s earlier and uses 22 % less torque energy on average.

2. **B: the paper's one-sided barrier is sensitive to the disturbance size; the symmetric barrier is not.** Up to the paper's disturbance amplitude every form works (100 % for both constrained forms in the 0.5–1× bin). Above it, the paper form drops to 23 % (1–1.5×) and 0 % beyond 1.5×, while the symmetric form stays at 96–97 % up to 3×. In the runs above 1.5×, *every* sample outside the envelope is at the *lower* wall of the rate envelope ($\omega_i < -\rho_\omega$), and all of them occur late ($t > 11\,\mathrm{s}$, when the tube is narrow), where the paper's barrier gain stays near $\beta/4$: exactly the one-sided property of section 3.5.

3. **C: no controller can beat physics.** With $J \approx 17\,\mathrm{kg\,m^2}$ and 2.5 N m the body can decelerate by only about $0.15\,\mathrm{rad/s^2}$ (Mathematical Models, block 1). An initial rate of 0.3 rad/s needs about 2 s to stop, while the rate envelope is already closing. Both constrained forms degrade identically with the initial rate (100 % below 0.12 rad/s, about 35 % at 0.33 rad/s): this is a feasibility limit of the envelope, not a controller defect.

5.2 Sensitivity to the parameters the paper does not give

Table 5: Constrained law (paper form), Part-2 initial state, different T_s and ρ_{q0}.

T_s (s)	ρ_{q0}	outside ρ_q	outside ρ_ω	torque at limit	max $\lvert\varepsilon_1\rvert$
15	0.36 (ours)	0.0 %	0.0 %	6.3 %	0.868
15	0.50	0.0 %	0.0 %	7.7 %	0.625
10	0.50	0.0 %	9.6 %	7.4 %	0.813
10	0.36	65.9 %	55.8 %	71.3 %	6.554

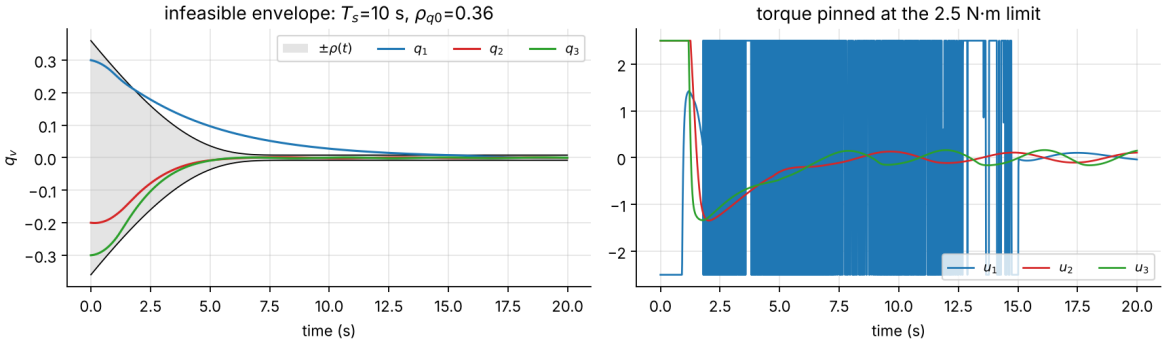


Figure 12: An infeasible envelope ($T_s = 10\,\mathrm{s}$ with the tight $\rho_{q0} = 0.36$, Part-2 state): the barrier asks for more torque than 2.5 N m, the torque stays pinned at the limit and the constraint is lost.

Lesson: the envelope must be feasible for the actuator

The paper presents ρ_0, ρ_T, T_s as free design choices "based on the requirements". They are not independent of $u_{\max}$: if the tube closes faster than the saturated actuator can turn the body, no gain can keep the state inside, and the barrier only drives the actuator into permanent saturation. This is the same limit as Monte-Carlo C, and it is the main design rule we took to the X3, whose inertia and torque authority are completely different.

5.3 Findings: what the paper does not say

#	Finding	How we handled it
P1	Four values are not given: c, ς, the Part-2 settling time and a usable Part-2 ρ_{q0}.	$c = 0.5$ (satisfies Theorem 1, $c < 0.964$), $\varsigma = 0.02$, $T_s = 15\,\mathrm{s}$, $\rho_{q0} = 0.36$; sensitivity measured (table 5).
P2	Part 2 starts on its own boundary ($\lvert q_1(0)\rvert = \lvert q_3(0)\rvert = 0.3 = \rho_{q0}$), so $(1-\varepsilon)^{-2} = \infty$ and Remark 7 is violated.	$\rho_{q0} = 1.2 \max \lvert q_i(0)\rvert = 0.36$.
P3	The barrier $(1-\varepsilon)^{-2}$ is one-sided.	Paper form is the reference; the symmetric form is reported as a variant; Monte-Carlo B shows the difference.
P4	Envelope parameters must be feasible for $u_{\max}$ and J.	Sensitivity study and Monte-Carlo C; used to size the X3 envelope.
P5	Theorem 1 proves convergence to the origin, not that $\lvert\varepsilon\rvert < 1$ for all t.	Constraint satisfaction is reported as measured, never as guaranteed.
P6	Eq. (15) is printed with $(1-q_0^2)$ and $\tilde{\theta}^2/\eta_1$; the derivative (17) follows from $(1-q_0)^2$ and $\frac{\eta_1}{2}\tilde{\theta}^2$.	Written in the consistent form (Mathematical Models, Theorem 1).

#	Finding	How we handled it
P7	With the boundary layer (22) and a leakage that vanishes, $\hat{\theta}$ slowly drifts upward.	Observed (fig. 10); on the X3 the leakage has a floor and $\hat{\theta}$ an upper bound.

6 Putting the Controller on the Gazebo X3 Quadcopter

6.1 Architecture

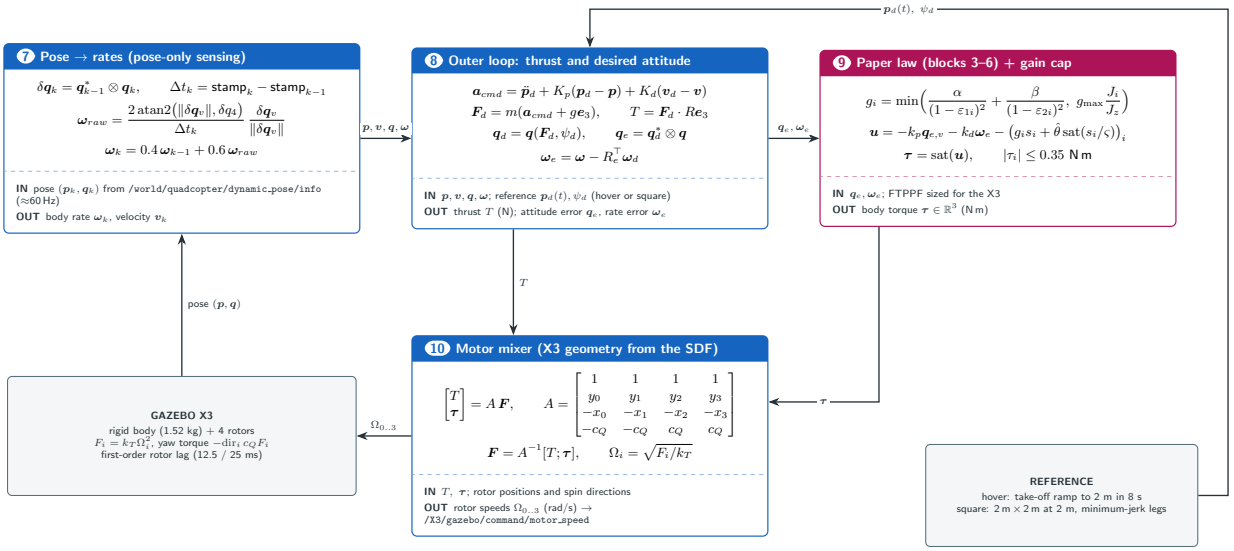


Figure 13: Implementation flow on the X3 (Mathematical Models, Section 4). Blocks 3–6 of the paper sit inside block 9.

Topic	Type	Use
`/world/quadcopter/dynamic_pose/info`	`geometry_msgs/PoseArray`	input: pose of the X3 with the simulation time stamp (≈60 Hz)
`/X3/gazebo/command/motor_speed`	`actuator_msgs/Actuators`	output: four rotor speeds in rad/s

Why a software-in-the-loop surrogate. Diagnosing a crash inside Gazebo is slow: one run per minute, no access to the internal state, no exact repetition. We therefore wrote `x3_sim/x3_model.py`, a model of the same vehicle in which *every constant is copied from the Gazebo model files* (mass, inertia, rotor positions and spin directions, thrust and moment constants, rotor time constants and speed limit, rotor drag), integrated with RK4 at 1 ms, sampled at the Gazebo pose rate of 60 Hz with one sample of transport delay. The controller sees only what the ROS node sees: a time stamp, a position and a quaternion. Both controllers are plain classes with the same `update(stamp, p, q)` interface, so the identical code runs in the surrogate, in the offline test and inside `gazebo.py`.

What the surrogate does not model

Rotor gyroscopic effects (rotor inertia is about 0.1 % of the body's), blade flapping, contacts other than a flat ground, sensor noise and the ROS/Gazebo transport beyond a fixed delay. Results on the surrogate are therefore "best case" for noise; Gazebo logs processed with `plot_gazebo_log.py` use the same metric definitions so the two can be compared line by line.

6.2 Why the first Gazebo node crashed

Our first node (`legacy/gazebo_v1.py`) made the X3 take off and climb, then lose its attitude and crash. To find out why without guessing, `OriginalGazeboController` is a line-by-line port of that node's control logic (only the ROS plumbing removed). On the surrogate it reproduces the observed behaviour, and each suspected fault can be switched off on its own (experiment E1).

#	Fault in v1	Consequence / fix in v2
F9	Mixer signs taken from a generic "+/−" template that does not match the X3 rotor numbering.	A pitch command produced roll torque (−0.93 per unit), nothing produced pitch torque (table 8). Fix: exact inverse of the allocation matrix built from the SDF (block 10).
F10	Yaw term divided by c_Q instead of $k_T c_Q$.	Yaw authority 10^5 times too small, yaw uncontrolled. Fix: same allocation matrix.
F14	`last_z` set once at start and never updated.	"Vertical speed" $= (z - z_0)/\Delta t$ grows with height, the altitude loop cuts thrust: the drone hops and falls. Fix: update every sample.
F11	Body rate from $2(\boldsymbol{q}_k - \boldsymbol{q}_{k-1})_v/\Delta t$ with a 0.10 filter.	Wrong axes when yawed, ≈150 ms lag; slow limit cycle. Fix: $\boldsymbol{q}_{k-1}^* \otimes \boldsymbol{q}_k$, filter 0.6 (block 7).
F12	Δt from the wall clock (`time.time()`).	Timing does not match simulation time when Gazebo runs slower or faster than real time. Fix: message stamp.
F13	Attitude error = raw world quaternion.	Works only for yaw 0 and no position control. Fix: $\boldsymbol{q}_e = \boldsymbol{q}_d^* \otimes \boldsymbol{q}$ with an outer position loop (block 8).
F3	Envelope effectively inactive ($\rho_{q0} = 2$ although $\lvert q_i \rvert \le 1$; $\alpha = \beta = 0.01$; $\boldsymbol{u}_{cons}$ clipped to 0.05 N m).	The first node was in practice a plain PD. Fix: envelope and gains sized for the X3 (block 9).
F15	(new in v2) unbounded barrier gain on a 60 Hz delayed loop.	Cap $g_i \le J_i/J_z$ (block 9; MATLAB step 8).

Table 8: Torque produced on the X3 per unit torque commanded by the v1 mixer (ideal: identity).

	cmd τ_x	cmd τ_y	cmd τ_z
produced τ_x	+0.044	**−0.933**	0
produced τ_y	0	**0**	7×10^{-5}
produced τ_z	+0.071	0	**0**

The two faults in the port of v1 that matter most, next to their fixes:

x3_sim/controllers.py — OriginalGazeboController (port of v1): z-rate and body rate

```
        raw_z_dot = (z - self.last_z) / dt
        self.z_dot = (1.0 - self.z_filter) * self.z_dot + self.z_filter * raw_z_dot
        if "zdot" in self.fixes:          # v1 never updates last_z (bug F14)
            self.last_z = z
        self.z_dot = np.clip(self.z_dot, -3.0, 3.0)
        if "omega" in self.fixes:
            d = quat_mul(quat_conj(self.last_q), q)
```

```
            raw_omega = 2.0 * d[:3] / dt
            omega = 0.4 * self.last_omega + 0.6 * raw_omega
        else:
            dq = (q - self.last_q) / dt
            raw_omega = 2.0 * dq[:3]
            omega = (1.0 - self.omega_filter) * self.last_omega + self.omega_filter * raw_omega
```

x3_sim/controllers.py — OriginalGazeboController (port of v1): the mixer

```
        base = T_total / (4.0 * self.c_T)
        roll_term = tau[0] / (4.0 * self.c_T * self.arm_length)
        pitch_term = tau[1] / (4.0 * self.c_T * self.arm_length)
        yaw_term = tau[2] / (4.0 * self.c_Q)
        m1 = base + pitch_term - roll_term - yaw_term
        m2 = base - pitch_term - roll_term + yaw_term
        m3 = base - pitch_term + roll_term - yaw_term
        m4 = base + pitch_term + roll_term + yaw_term
        w_cmd = np.sqrt(np.maximum(0.0, [m1, m2, m3, m4]))
```

6.3 The corrected controller (v2)

`FixedController` keeps the paper's law (Eq. 8, 11–14, 22) and adds blocks 7, 8, 10 and the gain cap of block 9.

x3_sim/controllers.py — block 7: exact body rate (F11)

```
        # ---- F11: exact body rate from consecutive quaternions -------------
        dq = quat_mul(quat_conj(self.last_q), q)
        if dq[3] < 0: dq = -dq
        ang = 2.0 * np.arctan2(np.linalg.norm(dq[:3]), dq[3])
        axis = dq[:3] / (np.linalg.norm(dq[:3]) + 1e-12)
        w_raw = axis * ang / dt
        self.omega = (1 - self.lam) * self.omega + self.lam * w_raw
        v_raw = (p - self.last_p) / dt
        self.vel = 0.7 * self.vel + 0.3 * v_raw
```

x3_sim/controllers.py — block 9: paper law with the sampled-data gain cap

```
        # ---- paper law, Eq. (8), (11)-(14), (22) ----------------------------
        rq = ftppf(t, e["rho_q0"], e["rho_qT"], e["kappa_q"], e["Ts"])
        rw = ftppf(t, e["rho_w0"], e["rho_wT"], e["kappa_w"], e["Ts"])
        eps1 = qv / rq; eps2 = w / rw
        s = w + g["c"] * qv
        u = -g["kp"] * qv - g["kd"] * w
        if self.mode in ("constrained", "symmetric"):
            if self.mode == "constrained":
                gain = g["alpha"] / (1 - np.minimum(eps1, 0.99)) ** 2 + g["beta"] / (1 - np.
    minimum(eps2, 0.99)) ** 2
            else:
                gain = g["alpha"] / (1 - np.clip(eps1, -0.99, 0.99) ** 2) ** 2 + \
                    g["beta"] / (1 - np.clip(eps2, -0.99, 0.99) ** 2) ** 2
            # sampled-data guard: the continuous-time barrier gain is unbounded as eps -> 1;
            # at 60 Hz with one sample of delay an unbounded gain destabilises the loop,
            # so the total barrier gain is capped at gain_max (see report, F15).
            gain = np.minimum(gain, g["gain_max"] * self.J_scale)
            u = u - (gain * s + self.theta * np.clip(s / g["varsigma"], -1, 1))
            eta2 = max(2.0 * np.exp(-0.5 * t), g["eta2_floor"])
            self.theta = float(np.clip(self.theta + dt * (np.linalg.norm(s) - eta2 * self.theta) /
     g["eta1"],
                                       0.0, g["theta_max"]))
        tau = np.clip(u, -self.uMax, self.uMax)
```

x3_sim/controllers.py — block 10: allocation with thrust-priority de-saturation (F9, F10)

```
        # ---- F9/F10: exact allocation with thrust-priority desaturation ----------
        F = self.Ainv @ np.array([T, *tau])
        Fmax = KT * W_MAX ** 2
        sat_frac = 0.0
        if F.min() < 0 or F.max() > Fmax:
            # drop yaw first, then scale roll/pitch until feasible
            F0 = self.Ainv @ np.array([T, tau[0], tau[1], 0.0])
            k = 1.0
            if F0.min() >= 0 and F0.max() <= Fmax:
                Fy = self.Ainv @ np.array([0, 0, 0, tau[2]])
                for kk in np.linspace(1, 0, 21):
                    if (F0 + kk * Fy).min() >= 0 and (F0 + kk * Fy).max() <= Fmax:
                        k = kk; break
                F = F0 + k * Fy
            else:
                Fh = self.Ainv @ np.array([T, 0, 0, 0]); Ft = F0 - Fh
                for kk in np.linspace(1, 0, 41):
                    if (Fh + kk * Ft).min() >= 0 and (Fh + kk * Ft).max() <= Fmax:
                        k = kk; break
                F = Fh + k * Ft
            sat_frac = 1.0
        F = np.clip(F, 0, Fmax)
        self.cmd = np.sqrt(F / KT)
```

Table 9: Parameters of the paper's law: spacecraft (paper) and X3 (ours).

Parameter	Paper (spacecraft)	X3
inertia	$J \approx 14$–$21\,\mathrm{kg\,m^2}$	$\mathrm{diag}(0.036, 0.070, 0.099)\,\mathrm{kg\,m^2}$
torque limit	$2.5\,\mathrm{N\,m}$	$0.35\,\mathrm{N\,m}$ (and rotor limits)
k_p, k_d	40, 20	4, 0.4
α, β, c	1.5, 1.2, 0.5	0.15, 0.12, 1.0
η_1, η_2	0.1, $2e^{-0.5t}$	5, $\max(2e^{-0.5t}, 0.2)$, $\hat{\theta} \le 0.1$
ς	0.02	0.5
ρ_q: $\rho_0 \to \rho_T$	$0.3 \to 0.008$	$0.30 \to 0.02$
ρ_ω: $\rho_0 \to \rho_T$	$0.4 \to 0.005\,\mathrm{rad/s}$	$2.5 \to 0.05\,\mathrm{rad/s}$
κ, T_s	2.5, 10–15 s	2, 10 s
natural frequency, damping (PD part)	$1.08\,\mathrm{rad/s}$, 0.54	$7.5\,\mathrm{rad/s}$, 0.75 (roll)
barrier gain cap	none	$g_i \le J_i/J_z$ (0.36, 0.71, 1.0)

6.4 The ROS 2 node

`gazebo.py` (v2) is a thin wrapper: it reads the pose and the *message stamp*, calls the `update()` method of `FixedController`, publishes the four rotor speeds, optionally injects the paper's disturbance as an extra body torque through the rotors, logs everything and stops after the chosen duration. The experiment, controller and disturbance are ROS parameters (section 2). The offline test drives the real node file, with `rclpy` and the two message types replaced by minimal stand-ins, against the surrogate at 60 Hz with one sample of delay.

In our project — offline test of the node (`tests/test_gazebo_node_offline.py`)

Hover, 40 s: final position (0.000, 0.000, 2.000) m. Square, 45 s: returns to (0.000, 0.000, 2.000) m. Hover with 0.5× the paper's disturbance injected through the rotors (symmetric barrier): final position (0.030, 0.056, 2.000) m. The log is processed by `plot_gazebo_log.py` without error. Result: `ALL OFFLINE NODE TESTS PASSED`.

7 Results on the X3

All experiments run on the X3 surrogate (section 6) at 60 Hz with one sample of delay, with a tiny asymmetric torque of 2×10^{-4} N m always applied so that a perfectly symmetric simulation cannot hide an instability. Numbers: `docs/results/x3_metrics.md`; script: `x3_sim/run_x3_experiments.py`.

Exp.	Question
E0/E1	Does the port of v1 crash like Gazebo did, and which fault causes it?
E2	Does the corrected paper controller hover for 180 s inside its envelopes?
E3	With the paper's disturbance (scaled to what the X3 can produce), constrained vs. conventional PD?
E3b	What happens at the full paper disturbance?
E4	Can it track a trajectory (2 m × 2 m square at 2 m)?
E5	How sensitive is it to control rate, delay and time-stamp jitter?

7.1 E0/E1: the crash of v1, reproduced and explained

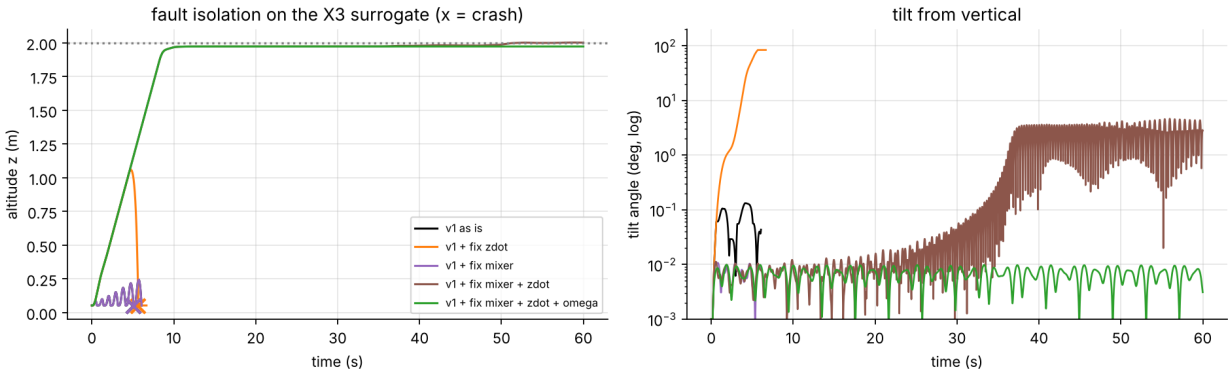


Figure 14: Fault isolation: v1 with each fix switched on separately (60 s, × = crash).

Table 10: E1 fault isolation (60 Hz).

Variant	crashed	crash time	max altitude	RMS tilt after 20 s
v1 as is	yes	5.07 s	0.24 m	–
v1 + z-rate fix	yes	5.68 s	1.06 m	–
v1 + mixer fix	yes	5.07 s	0.24 m	–
v1 + mixer + z-rate fixes	no	–	2.00 m	2.03°
v1 + mixer + z-rate + body-rate fixes	no	–	1.97 m	0.007°
v1 as is at 250 Hz	yes	14.67 s	0.15 m	–

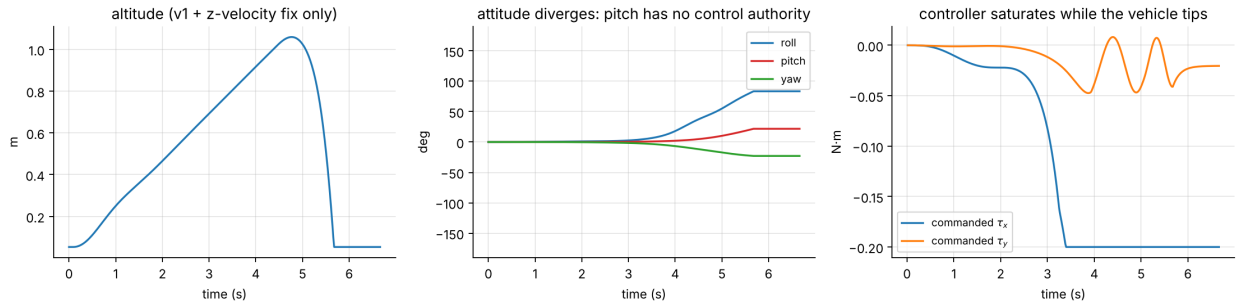


Figure 15: v1 with only the z-rate fixed: the behaviour we saw in Gazebo. It climbs, then the attitude diverges because pitch has no authority, the commanded roll torque saturates, and the vehicle crashes at 5.7 s.

In our project — conclusion of E1

Two faults are each sufficient to crash the vehicle: the altitude-rate bug F14 (it never climbs above 0.24 m) and the mixer F9 (with F14 fixed it climbs to 1.06 m and then tips over, exactly as observed in Gazebo). Fixing both gives a stable flight; the remaining slow attitude limit cycle (2° RMS) comes from the lagging body-rate estimate F11 and disappears with the exact estimate. A faster loop (250 Hz) does not save v1: the faults are structural, not a matter of rate.

7.2 E2: 180 s hover with the paper's law

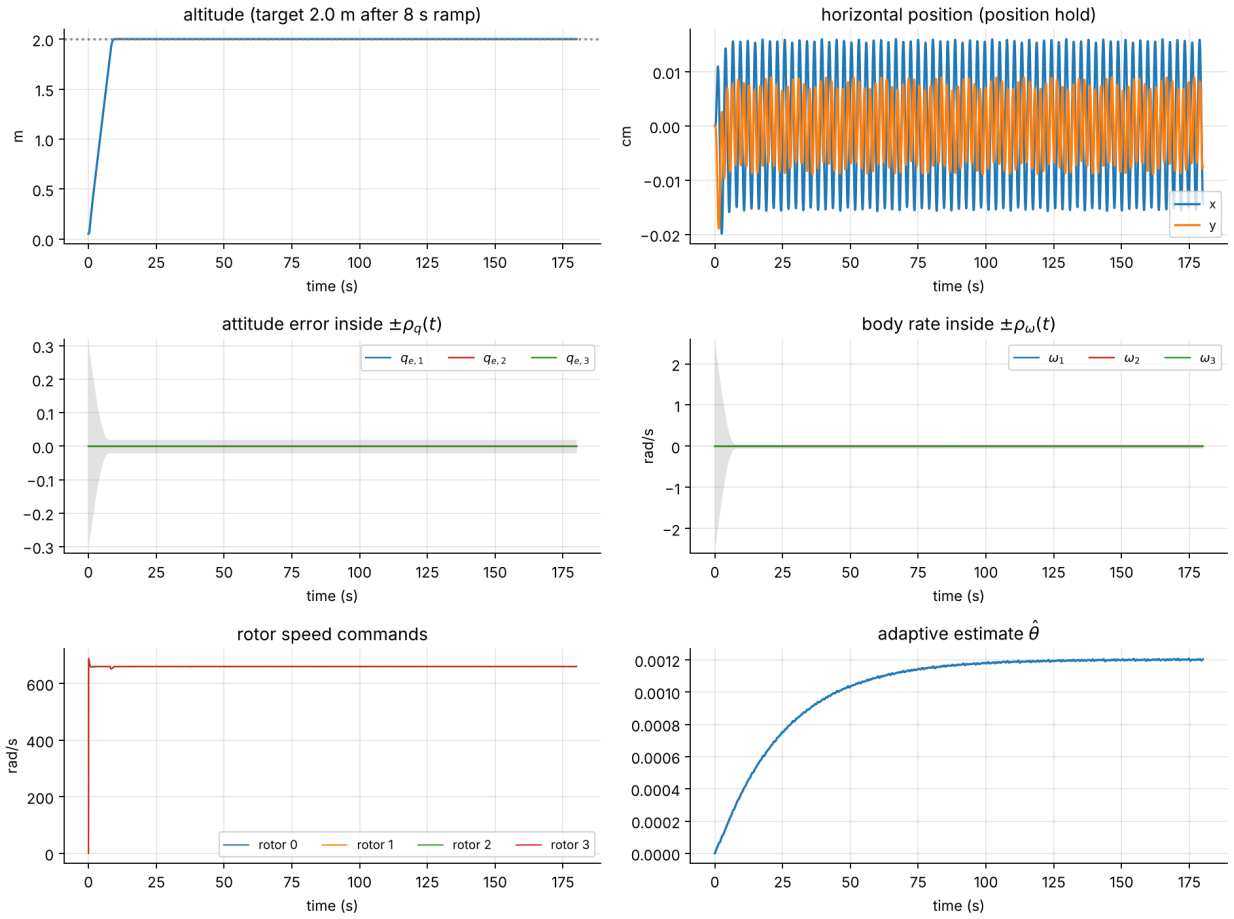


Figure 16: E2, corrected controller (paper barrier form), 180 s: altitude, horizontal position, attitude error and body rate inside their envelopes, rotor commands and $\hat{\theta}$.

Table 11: E2 metrics (after the 8 s take-off ramp where stated).

Metric	Value	Metric	Value
crashed	no	time outside ρ_q / ρ_ω	0 % / 0 %
altitude RMS error ($t \geq 10$ s)	0.02 mm	max $\lvert\varepsilon_1\rvert$ / max $\lvert\varepsilon_2\rvert$	0.0023 / 0.0048
altitude max error ($t \geq 10$ s)	0.24 mm	mixer saturated	0 % of the time
horizontal drift (max)	0.21 mm	rotor speeds ($t \geq 10$ s)	659.9–660.7 rad/s
tilt RMS ($t \geq 10$ s)	0.003°	$\hat{\theta}$ at 180 s	0.0012 N m

How to read these very small numbers

The surrogate has no sensor noise and only the tiny test torque, so these are the errors of the control loop itself (it is stable, well damped and has no limit cycle), not predictions of the errors in Gazebo or on hardware. The rotor speeds sit at the hover value 660.4 rad/s computed by hand in the Mathematical Models (block 10).

7.3 E3: disturbance rejection, constrained vs. conventional PD

The paper's disturbance has a yaw amplitude of 0.15 N m, while the X3 can produce at most 0.106 N m of yaw torque at hover. E3 therefore uses 0.5× the paper's $\boldsymbol{d}(t)$, applied from $t = 15$ s to 180 s (E3b uses 1×).

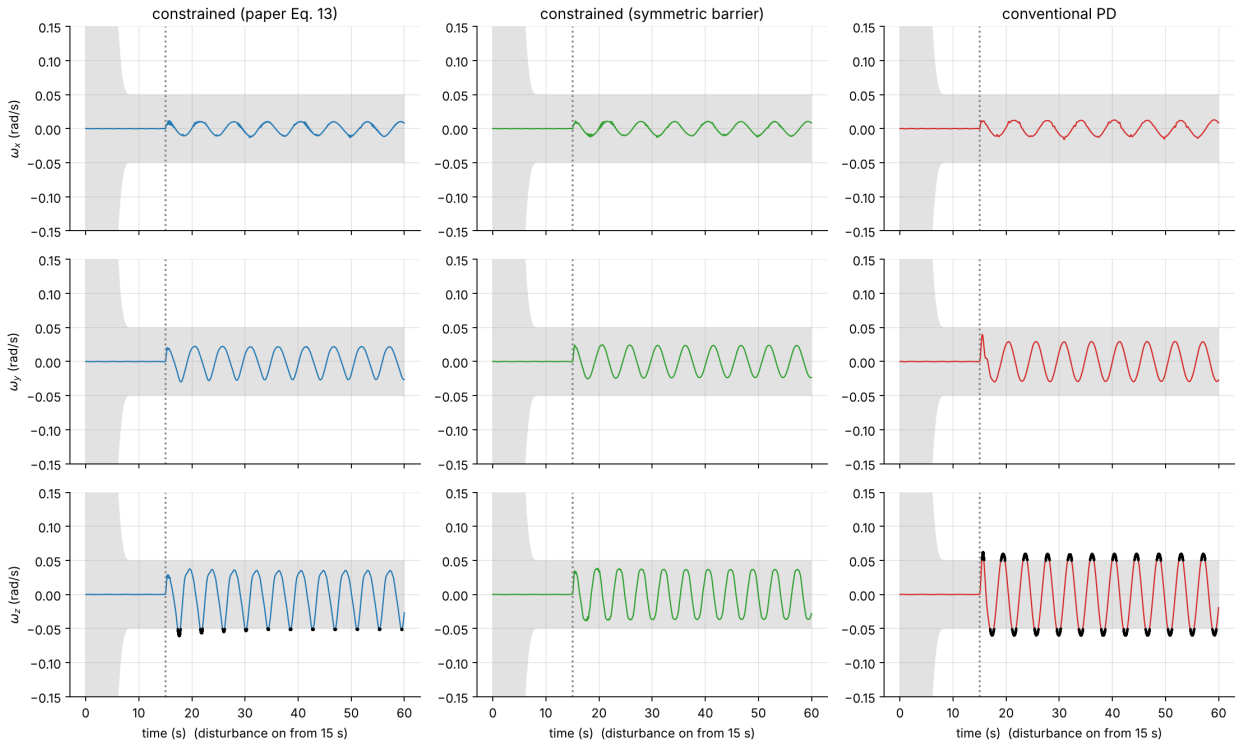


Figure 17: E3, body rates against the rate envelope ($\rho_{\omega T} = 0.05$ rad/s), first 60 s. Black: outside. The paper form touches the *lower* yaw wall, the symmetric form never leaves, conventional PD leaves at both walls on every yaw oscillation.

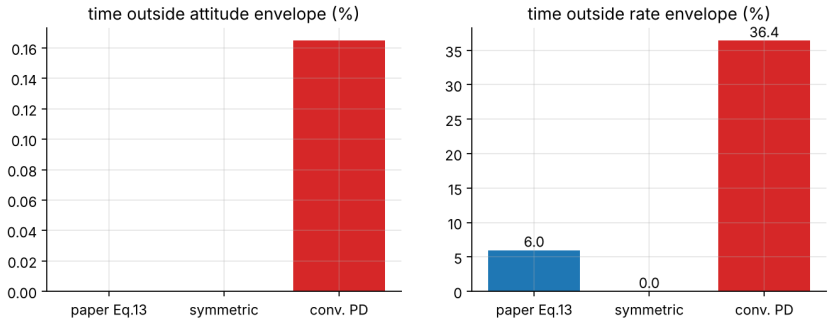


Figure 18: E3, percentage of time outside each envelope (15–180 s).

Table 12: E3 metrics, disturbance 0.5× paper $\boldsymbol{d}(t)$, $t = 15$–180 s.

Metric	constrained (paper)	constrained (symm.)	conventional PD
crashed	no	no	no
time outside ρ_q	0 %	0 %	0.16 %
time outside ρ_ω	6.0 %	**0 %**	36.4 %
$\max\lvert\varepsilon_1\rvert$ / $\max\lvert\varepsilon_2\rvert$	0.93 / 1.21	0.72 / 0.77	1.02 / 1.25
tilt RMS	0.43°	0.43°	0.52°
yaw RMS	1.16°	1.06°	1.61°
max horizontal error	12.3 cm	13.1 cm	14.9 cm

In our project — conclusion of E3

On the quadcopter the paper's qualitative claim holds again: with the same gains and limits, the constrained law cuts the time outside the rate envelope from 36.4 % (PD) to 6.0 % (paper form) and to 0 % (symmetric form), and reduces the tilt and yaw errors. The paper form's remaining violations are all at the lower yaw wall, the one-sided effect seen in the spacecraft Monte-Carlo study B.

E3b: full paper disturbance

At 1× the paper's disturbance the yaw torque demanded exceeds what the X3's rotors can produce while hovering (0.15 vs. 0.106 N m): the mixer is saturated 97 % of the time and all three controllers are outside the envelopes 96–98 % of the time (none crashes; thrust has priority). This is the feasibility limit of section 5.2 in physical form: no controller can satisfy a constraint the actuators cannot.

7.4 E4: square trajectory

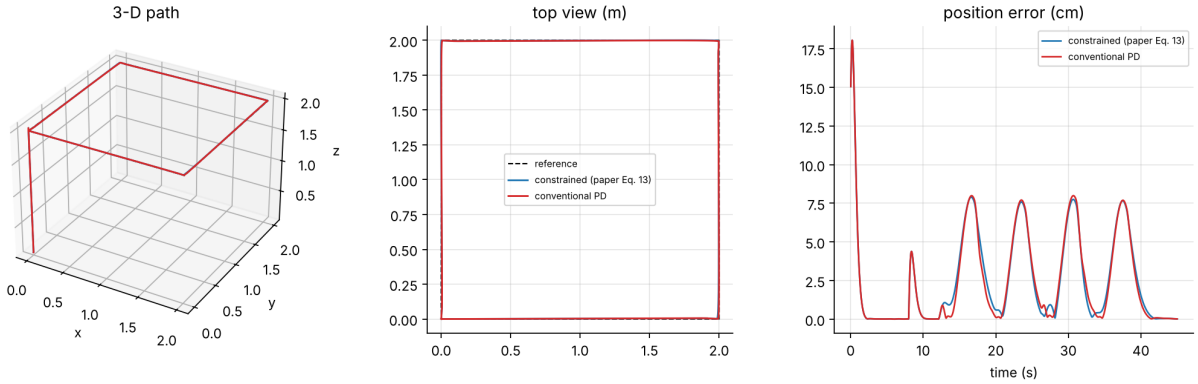


Figure 19: E4, 2×2 m square at 2 m altitude with minimum-jerk legs of 5 s and 2 s holds. Left: 3-D path; middle: top view; right: position error (the first second is the take-off from the ground).

Table 13: E4 metrics (after the take-off, $t \geq 12$ s).

Metric	constrained (paper)	conventional PD
crashed	no	no
RMS / max position error	4.03 / 7.90 cm	4.02 / 7.99 cm
time outside ρ_q	0 %	0 %
time outside ρ_ω	4.7 %	8.1 %
max $\lvert\varepsilon_1\rvert$	0.30	0.36

In our project — conclusion of E4

The paper's attitude law, with our outer loop, flies the square with 4 cm RMS error and never leaves the attitude envelope. Position tracking is set by the outer loop and is the same for both controllers. During the turns at the corners the required roll and pitch rates exceed the hover-sized rate envelope ($\rho_{\omega T} = 0.05$ rad/s): the constrained law halves the time outside it (4.7 vs. 8.1 %) but cannot remove it. For trajectory flight the rate envelope should be widened to the rates the trajectory needs (feasibility again).

7.5 E5: control rate, delay and jitter

Table 14: E5, disturbance 0.5×, 90 s; time outside ρ_q / ρ_ω in %. No run crashed.

Loop	constrained (paper)	constrained (symm.)	conventional PD
60 Hz, 1 sample delay (nominal)	0.0 / 6.6	0.0 / 0.0	0.4 / 36.2
100 Hz, 1 sample delay	0.0 / 6.8	0.0 / 0.0	0.4 / 36.7
250 Hz, 1 sample delay	0.0 / 7.1	0.0 / 0.0	1.5 / 36.9
60 Hz, 2 samples delay	18.0 / 90.6	20.7 / 90.6	0.4 / 36.3
60 Hz, 2 ms stamp jitter	0.0 / 21.6	0.0 / 50.2	0.3 / 44.3
30 Hz, 1 sample delay	52.8 / 96.9	94.2 / 99.8	68.5 / 99.2

What E5 tells us about deployment

At 60 Hz and faster the result is unchanged. Two things degrade the constrained laws far more than plain PD: a second sample of delay (rate-envelope violations rise from 6.6 to 90.6 % for the paper form, while PD stays at 36 %) and 2 ms of jitter on the time stamps (6.6 to 21.6 % for the paper form, 0 to 50.2 % for the symmetric form, 36 to 44 % for PD). Both act through the body-rate estimate, which the barrier amplifies near the wall (even with the cap). At 30 Hz no controller keeps the envelopes. In Gazebo the stamp must therefore be the simulation time of the message (F12), and the pose rate should be at least 60 Hz.

8 Visual Demonstrations (MuJoCo and Browser)

These demos make the controller *visible*; the quantitative evidence is in sections 4, 5 and 7.

8.1 MuJoCo: two bodies side by side

`Drones_Mujoco_Pointing_Trial.ipynb` integrates the paper's equations for two rigid bodies in MuJoCo's 3-D viewer: the left (blue) one runs the constrained PD-type law (paper barrier form), the right (orange) one conventional PD. Three demos are included: the paper's Part 2 manoeuvre, turning to any attitude entered as roll/pitch/yaw, and pointing the nose at a target position (each body's dotted line of sight reaches the gold target only when it is truly aimed at it). The notebook sizes ρ_{q0} from the requested rotation so that the initial state is strictly inside the envelope (e.g. $\rho_{q0} = 0.367$ for a $30°/30°/30°$ target, 1.057 for the large pointing turn).

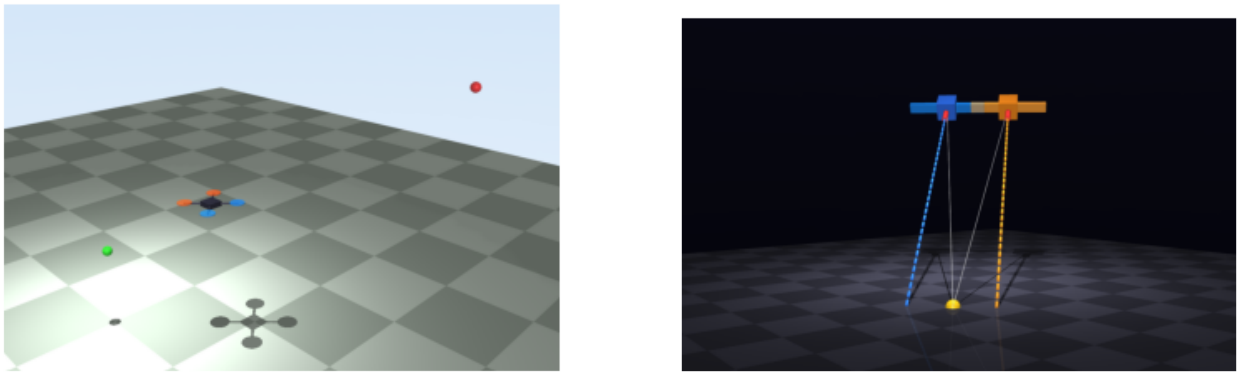


Figure 20: MuJoCo scenes from the notebook: a quadrotor-shaped body flying to a goal (left) and the two bodies pointing their lines of sight at a target (right).

Toy example — printed output of the Part-2 demo, $\|q_v\|$ every second

t (s)	0	1	2	3	4	5	6	7	8	9
constrained	0.469	0.401	0.240	0.128	0.067	0.036	0.017	0.009	0.005	0.002
conventional PD	0.469	0.401	0.214	0.031	0.048	0.040	0.016	0.003	0.007	0.006

The constrained body decays monotonically (no overshoot through zero); the PD body is sometimes ahead and then swings past zero and back ($0.031 \to 0.048$). $\|\boldsymbol{q}_v\|$ alone does not show the envelope, which is why the reproduction reports $|\varepsilon|$ and the time outside (section 4.2).

8.2 MuJoCo: flight with altitude and attitude loops

`Drones_Mujoco_Flying.ipynb` and `Drones_Mujoco_Good_Trial.ipynb` add an altitude/position loop around the attitude law on a quadrotor-shaped body with direct force and torque actuators (not four-rotor mixing).

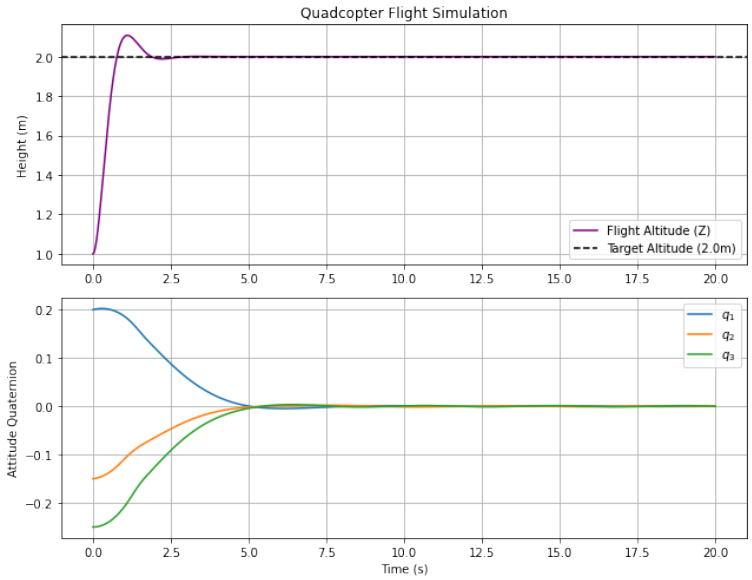


Figure 21: MuJoCo flight: climb to 2 m while the attitude quaternion converges from the paper's Part-1 initial attitude.

The point-to-point flight from $(0, 0, 1)$ to $(3, 3, 3)$ (minimum-jerk reference over 6 s, then 3 s hover) prints its position every 0.5 s and ends at $(3.01,\ 3.01,\ 3.00)$ m (`Drones_Mujoco_Good_Trial.ipynb`). The flying notebook also keeps our earlier unsuccessful attempts (wrong sign conventions in the position loop, cells 6–8) as development history; they are not used as results.

8.3 Interactive browser simulator

`index.html` (with `app.js`, `simulation.js`) runs the rigid-body equations with RK4 in the browser and plots quaternion, rate, torque and $\hat{\theta}$ live for the constrained law and PD. It uses the symmetric barrier form and is meant for illustration in a presentation; it is not used for any number in this report.

9 The MATLAB Toy Model

The toy model is a set of small MATLAB scripts that make every block of the controller tangible: each one prints its inputs and outputs with their sizes, units and meaning, reproduces the hand-calculated toy examples of the Mathematical Models to six digits, and draws one figure that shows the idea. No toolbox is needed.

9.1 The files

File in `toy_model/`	Blocks	What you see when you run it
`step1_ftppf.m`	3	$\rho_q(2)$ by hand; $\rho(0) = \rho_0$, $\rho(T_s) = \rho_T$, monotonicity; FTPPF vs. exponential PPF; effect of T_s and κ
`step2_normalised_error.m`	4, 5	$\boldsymbol{\varepsilon}_1$, $\boldsymbol{\varepsilon}_2$ of the Part-1 state; barrier gains of both forms; the one-sided property
`step3_control_law.m`	5, 2	$\boldsymbol{s}$, $\boldsymbol{u}_{PD}$, $\boldsymbol{u}_{cons}$, $\boldsymbol{u}$, sat($\boldsymbol{u}$) at $t = 0$; the three controllers on the same state
`step4_adaptive_law.m`	6	first Euler step by hand; $\hat{\theta}$ for a decaying and a persistent $\|\boldsymbol{s}\|$; drift and the leakage floor
`step5_single_axis.m`	1–6	the whole idea on one axis ($J\ddot{x} = \text{sat}(u) + d$): constrained vs. PD with the same gains and limit
`step6_spacecraft_closed_loop.m`	1–6	independent re-implementation of the paper's Part 1 and Part 2 (cross-check of the Python numbers)
`step7_x3_mixer.m`	10	X3 allocation matrix from the SDF, hover speeds, torque authority, the v1 mixer error
`step8_sampled_data_limit.m`	9	why the barrier gain must be capped on a 60 Hz loop with one sample of delay
`run_all.m`	all	runs steps 1–8, saves `figures/*.png` and the complete printed output in `results_log.txt`
`lib/`	–	`ftppf.m`, `control_law.m`, `paper_params.m`, `io_print.m`, `step_header.m`

9.2 How to run them

1. Open MATLAB and run `run('<repo>/toy_model/run_all.m')`, or open one step and press *Run*. Every step adds `lib/` to the path itself, so it can be run on its own from any folder.
2. Each step starts with a banner: step number, title and the equations of the block. Every printed line is tagged `[IN ]`, `[OUT]` or `[CHK]` and shows name, size, unit and meaning, followed by the value.
3. The figures are written to `toy_model/figures/`; this document includes them directly from there.

9.3 What the steps show

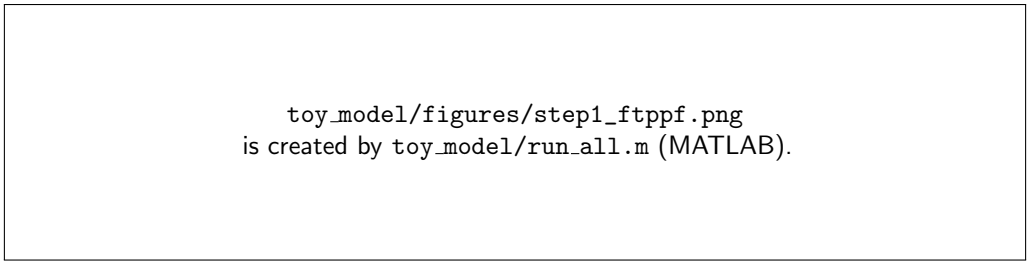


Figure 22: Step 1: the finite-time property and the effect of T_s and κ. *(MATLAB toy model, toy_model/figures/step1_ftppf.png.)*

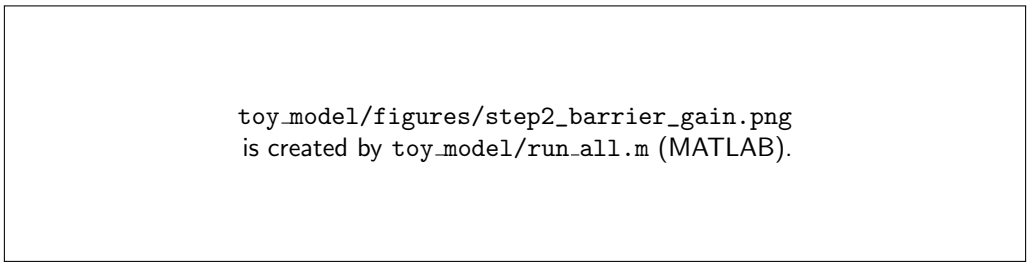


Figure 23: Step 2: barrier gain of the paper form and the symmetric form, and the extra restoring torque for $s = 0.05$. *(MATLAB toy model, toy_model/figures/step2_barrier_gain.png.)*

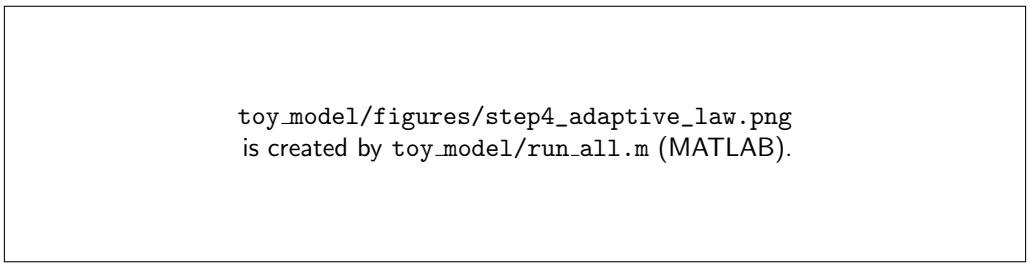


Figure 24: Step 4: $\hat{\theta}$ for a decaying $\|\boldsymbol{s}\|$, for a small persistent residual (drift), and with a leakage floor $\eta_2 \geq 0.2$. *(MATLAB toy model, toy_model/figures/step4_adaptive_law.png.)*

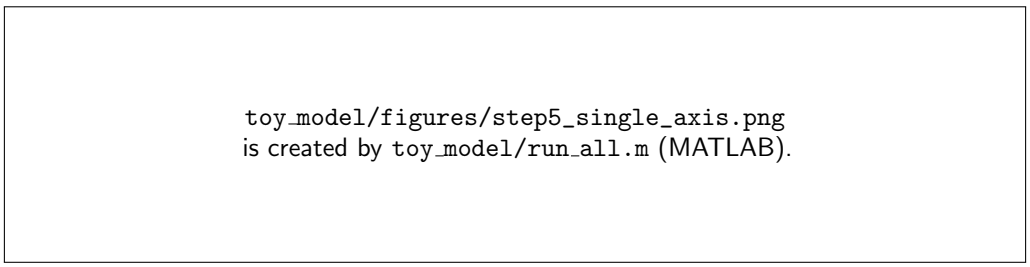


Figure 25: Step 5: one axis, $J = 1$, $|u| \leq 3$, $d = 0.3 \sin 2t$. Same gains for both controllers; only the constrained one stays inside the tube. *(MATLAB toy model, toy_model/figures/step5_single_axis.png.)*

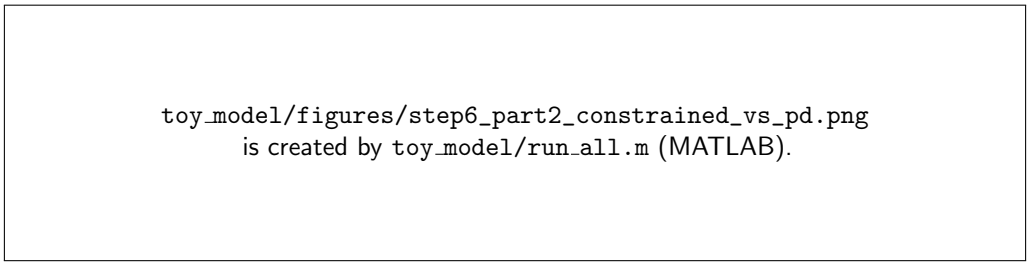


Figure 26: Step 6: MATLAB re-implementation of the paper's Part 2. *(MATLAB toy model, toy_model/figures/step6_part2_constrained_vs_pd.png.)*

toy_model/figures/step6_part2_eps2.png
is created by toy_model/run_all.m (MATLAB).

Figure 27: Step 6: $\max_i |\varepsilon_{2i}|$ for the three controllers (MATLAB). *(MATLAB toy model, toy_model/figures/step6_part2_eps2.png.)*

toy_model/figures/step7_x3_mixer.png
is created by toy_model/run_all.m (MATLAB).

Figure 28: Step 7: torque produced per unit commanded by the v1 mixer and by the corrected mixer. *(MATLAB toy model, toy_model/figures/step7_x3_mixer.png.)*

toy_model/figures/step8_sampled_data_limit.png
is created by toy_model/run_all.m (MATLAB).

Figure 29: Step 8: spectral radius of the sampled roll, pitch and yaw loops against the total gain on the rate. *(MATLAB toy model, toy_model/figures/step8_sampled_data_limit.png.)*

9.4 Printed output

`run_all.m` writes the complete printed output to `toy_model/results_log.txt`.

9.5 Things to try

- In `step5`, set `S.alpha = 0; S.beta = 0;` to remove the barrier and compare the time outside the tube with the constrained run.
- In `step6`, set `P2.rq(1) = 0.3` (the paper's own Part-2 value): $\varepsilon_{11}(0) = 1$ and the barrier gain is only finite because of the 0.999 guard ($1.5/0.001^2 = 1.5 \times 10^6$), finding P2.
- In `step6`, lower `P.umax` and watch when the envelope stops being feasible (section 5.2).
- In `step8`, change `T = 1/60` to `1/250` and see how the stable range of the total gain grows.

10 Conclusions and Scope of the Claims

10.1 What we have shown

Claim	Evidence
Every equation of the paper (1)–(14), (22) is implemented and traceable to code.	Mathematical Models Section 6; section 3.
The paper's Part 1 is reproduced: both states inside their envelopes for $T_s = 15$ and 10 s, faster convergence with the smaller T_s, torque within the limit.	table 1, fig. 4.
The paper's Part 2 is reproduced: at the same 2.5 N m limit the constrained law stays inside both envelopes, conventional PD leaves the rate envelope 27.5 % of the time; the constrained law settles earlier with half the residual error and 36 % less torque energy.	table 2, figs. 7 and 9.
The result is general within the paper's regime: 97.0 % violation-free over 300 random starts (PD: 0 %).	table 4, study A.
Every toy example and every block can be re-run step by step in MATLAB, including an independent re-implementation of the spacecraft experiments.	section 9, table 3.
The paper's barrier is one-sided; above the paper's disturbance level this costs constraint satisfaction, and a symmetric barrier removes the problem (97 % up to 3× the disturbance).	section 3.5, study B, X3 experiment E3.
The envelope must be feasible for the actuator; otherwise no controller can keep it.	table 5, study C, E3b.
Our first Gazebo node crashed because of a mixer that did not match the X3 and an altitude-rate bug; both are identified by controlled experiments and fixed.	table 10, table 8.
The paper's law, re-sized for the X3 and with a sampled-data gain cap, hovers for 180 s and flies a 2 m square (4 cm RMS) with 0 % attitude-envelope violations on a software-in-the-loop X3 built from Gazebo's model files; under disturbance it cuts rate-envelope violations from 36 % (PD) to 6 % (paper form) or 0 % (symmetric form).	tables 11 to 13.
The ROS 2 node runs exactly this controller and passes the offline test against the surrogate.	section 6, `tests/`.

10.2 What we do not claim

- **A proof that the envelope is never left.** The paper's Theorem 1 proves convergence; constraint satisfaction is measured, and it does fail where the envelope is infeasible or the disturbance exceeds what the one-sided barrier handles.
- **Number-for-number agreement with the paper's figures.** The paper publishes plots only and leaves four values unstated; we match its behaviour and report our own numbers with the choices disclosed.
- **Results from a Gazebo log of the v2 node.** The X3 numbers in this document come from the software-in-the-loop surrogate (Gazebo's constants, 60 Hz, one sample of delay, no sensor noise). Running `gazebo.py` in Gazebo and processing the log with `plot_gazebo_log.py` produces the same figures and metrics from the simulator itself (section 2).

- **Hardware flight.** No physical vehicle was flown.

10.3 Conclusion

The constrained PD-type controller of Golestani et al. does what the paper says in the setting the paper uses: with the same gains and the same saturated actuator as a conventional PD, it keeps both the attitude and the angular velocity inside finite-time prescribed envelopes, converges faster and with less control effort, and it does so in 97 % of random initial conditions, not only in the two cases shown in the paper. Its structure is as simple as claimed: our whole implementation of the law is about twenty lines. Carrying it onto a quadcopter required three additions the paper does not need (body rates from poses, an outer position loop, a correct motor mixer) and one change it does not anticipate (a cap on the barrier gain for a 60 Hz digital loop); with them the law flies the X3 model in hover and along a trajectory. Two lessons go beyond the paper: the envelope must be chosen to be feasible for the actuator, and a two-sided barrier is markedly more robust than the one-sided form printed in the paper.

10.4 Future work

1. Gazebo campaign with `gazebo.py`: hover, square and disturbance runs for the three controllers, compared with the surrogate numbers of section 7.
2. An envelope-feasibility rule: choose ρ_0, ρ_T, T_s from $u_{\max}/\lambda_{\max}(J)$ and the expected disturbance.
3. Rate-envelope scheduling for trajectory flight (widen ρ_ω during manoeuvres).
4. Actuator rate limits, which the paper itself names as future work.

A Appendix

A.1 Where every figure and number comes from

Output	Written by	Used in
`fig01_ppf_comparison.png`	`paper_reproduction.py`	fig. 2
`fig02_part1_Ts15/Ts10.png`, `fig03`	`paper_reproduction.py`	section 4.1
`fig04`–`fig08`	`paper_reproduction.py`	section 4.2
`fig09_barrier_forms.png`	`paper_reproduction.py`	section 3.5
`fig10`, `fig11`	`paper_reproduction.py`	section 5
`results/metrics.md`, `.json`	`paper_reproduction.py`	sections 4 and 5
`x3_E0`–`x3_E4` figures	`x3_sim/run_x3_experiments.py`	section 7
`results/x3_metrics.md`, `.json`	`x3_sim/run_x3_experiments.py`	sections 6 and 7
`toy_model/figures/*.png`, `results_log.txt`	`toy_model/run_all.m` (MATLAB)	sections 4.4 and 9
`mujoco_*.png`	MuJoCo notebooks (outputs exported)	section 8
`flow_paper.pdf`, `flow_x3.pdf`	`mathematical_models/figures/*.tex`	figs. 1 and 13
`gazebo_<mode>_<exp>.png/.json`	`plot_gazebo_log.py` from a Gazebo run	–

A.2 All parameters in one place

Group	Values
Spacecraft (paper)	$J = [20, 1.2, 0.9;\ 1.2, 17, 1.4;\ 0.9, 1.4, 15]\,\mathrm{kg\,m^2}$; $u_{\max} = 2.5\,\mathrm{N\,m}$; $\boldsymbol{d}(t) = [0.05\sin t,\ 0.1\sin 1.2t,\ 0.15\sin 1.5t]\,\mathrm{N\,m}$
Law (paper)	$k_p = 40$, $k_d = 20$, $\alpha = 1.5$, $\beta = 1.2$, $\eta_1 = 0.1$, $\eta_2 = 2e^{-0.5t}$, $\hat{\theta}(0) = 0$
Envelope (paper)	$\rho_q : 0.3 \to 0.008$, $\rho_\omega : 0.4 \to 0.005\,\mathrm{rad/s}$, $\kappa = 2.5$, $T_s = 15$ or $10\,\mathrm{s}$
Unstated (ours)	$c = 0.5$, $\varsigma = 0.02$, Part 2: $T_s = 15\,\mathrm{s}$, $\rho_{q0} = 0.36$
Simulation	RK4, 1 ms, ZOH torque, 20 s; Monte-Carlo $N = 300$, seed 7
X3 vehicle	$m = 1.52\,\mathrm{kg}$, $J = \mathrm{diag}(0.03565, 0.07035, 0.09892)\,\mathrm{kg\,m^2}$, $k_T = 8.54858\times10^{-6}$, $c_Q = 0.016$, $\Omega_{\max} = 800\,\mathrm{rad/s}$
X3 law	$k_p = 4$, $k_d = 0.4$, $\alpha = 0.15$, $\beta = 0.12$, $c = 1$, $\varsigma = 0.5$, $\eta_1 = 5$, $\eta_2 \geq 0.2$, $\hat{\theta} \leq 0.1$, $g_{\max} = 1$, $\lvert\tau\rvert \leq 0.35\,\mathrm{N\,m}$
X3 envelope	$\rho_q : 0.30 \to 0.02$, $\rho_\omega : 2.5 \to 0.05\,\mathrm{rad/s}$, $\kappa = 2$, $T_s = 10\,\mathrm{s}$
X3 outer loop	$K_{p,z} = 5$, $K_{d,z} = 3.5$, $K_{p,xy} = 1.2$, $K_{d,xy} = 1.6$, tilt $\leq 0.35\,\mathrm{rad}$; rate filter 0.6, velocity filter 0.3

A.3 References

1. M. Golestani, K. A. Alattas, S. Ud Din, S. Mobayen, A. K. Alanazi, A. Fekih, "A Simple Structure Constrained Attitude Control for Rigid Bodies: A PD-Type Control," *IEEE Access*, vol. 10, pp. 10202–10209, 2022, doi:10.1109/ACCESS.2022.3144627.
2. C. P. Bechlioulis, G. A. Rovithakis, "Robust adaptive control of feedback linearizable MIMO nonlinear systems with prescribed performance," *IEEE Trans. Autom. Control*, vol. 53, no. 9, pp. 2090–2099, 2008.
3. J. T.-Y. Wen, K. Kreutz-Delgado, "The attitude control problem," *IEEE Trans. Autom. Control*, vol. 36, no. 10, pp. 1148–1162, 1991.
4. H. K. Khalil, *Nonlinear Systems*, 3rd ed., Prentice Hall, 2002.
5. Z. Chen, J. Huang, "Attitude tracking and disturbance rejection of rigid spacecraft by adaptive control," *IEEE Trans. Autom. Control*, vol. 54, no. 3, pp. 600–605, 2009. (Source of the paper's inertia matrix.)
6. Gazebo Sim: `quadcopter.sdf` example world and the `MulticopterMotorModel` system.
7. E. Todorov, T. Erez, Y. Tassa, "MuJoCo: A physics engine for model-based control," IROS 2012.